

دانشگاه تهران

دانشکده فنی

دانشکده مهندسی برق و کامپیوتر



# گزارش تمرین کامپیوتری

اصول سیستم‌های مخابراتی

طراح پروژه:

امیرپرهم فارسی

دانشجو:

پریا خلیلی

شماره دانشجویی: ۸۱۰۸۰۱۰۶۵

نیمسال دوم ۱۴۰۵-۱۴۰۶

## فهرست مطالب

|    |                                                    |
|----|----------------------------------------------------|
| ۳  | چکیده                                              |
| ۴  | ۱ مقدمه                                            |
| ۵  | ۱.۱ سوال ۱: نمایش سیگنال‌ها در حوزه زمان           |
| ۸  | ۲.۱ سوال ۲: طیف دامنه و فاز (fft و fftshift)       |
| ۱۲ | ۳.۱ سوال ۳: زوم روی باند فرکانسی غالب              |
| ۱۷ | ۴.۱ سوال ۴: مقایسه با صدای شخصی ضبط شده            |
| ۲۱ | ۲ مدولاسیون DSB-SC                                 |
| ۲۱ | ۱.۲ تنظیمات شبیه‌سازی                              |
| ۲۲ | ۲.۲ دمودولاسیون و فیلتر پایین‌گذر ایده‌آل          |
| ۲۳ | ۳.۲ سوال ۱: نمایش سیگنال‌ها و محاسبه MSE           |
| ۲۵ | ۴.۲ سوال ۲: محاسبه SNR خروجی                       |
| ۲۵ | ۵.۲ سوال ۳: بررسی اثر اختلالات                     |
| ۲۵ | ۱.۵.۲ (الف) اختلاف فاز اسیلاتور                    |
| ۲۷ | ۲.۵.۲ (ب) اختلاف فرکانس اسیلاتور                   |
| ۲۹ | ۳.۵.۲ (ج) فیلتر باترورث به جای فیلتر ایده‌آل       |
| ۳۲ | ۶.۲ سوال ۴: راه‌حل مهندسی برای اختلاف فاز و فرکانس |
| ۳۷ | ۳ مدولاسیون SSB                                    |
| ۳۷ | ۱.۳ پیاده‌سازی دستی تبدیل هیلبرت                   |
| ۳۸ | ۲.۳ سوال ۱: نمایش سیگنال‌های مدوله و بازیابی‌شده   |
| ۴۰ | ۳.۳ سوال ۲ و ۳: محاسبه MSE و SNR                   |
| ۴۰ | ۴.۳ تکرار سوال ۳: تحلیل اختلالات برای SSB          |
| ۴۰ | ۱.۴.۳ (الف) اختلاف فاز اسیلاتور                    |
| ۴۱ | ۲.۴.۳ (ب) اختلاف فرکانس اسیلاتور                   |
| ۴۲ | ۳.۴.۳ (ج) فیلتر باترورث به جای LPF ایده‌آل         |
| ۴۸ | ۴ مدولاسیون FM (Simulink)                          |

|    |                                                                   |     |
|----|-------------------------------------------------------------------|-----|
| ۴۸ | سوال ۱: مدولاتور مبتنی بر بلوک VCO                                | ۱.۴ |
| ۴۹ | سوال ۲: جایگزینی آشکارساز فاز با بلوک ضرب‌کننده (Matrix Multiply) | ۲.۴ |
| ۵۰ | سوال ۳: ساختار مدل و نتایج                                        | ۳.۴ |
| ۵۴ | سوال ۴: رابطه Carson                                              | ۴.۴ |
| ۵۵ | سوال ۵: برآورد SNR از روی Spectrum Analyzer                       | ۵.۴ |

## چکیده

در این گزارش، سه روش مدولاسیون آنالوگ خطی و غیرخطی - DSB-SC، SSB و FM - به صورت عملی پیاده‌سازی و بررسی شده‌اند. برای دو مدولاسیون خطی (SSB و DSB)، پیاده‌سازی کامل مدولاتور و دمودولاتور در نرم‌افزار MATLAB انجام شده است، به گونه‌ای که فیلتر پایین‌گذر گیرنده به صورت ایده‌آل و از طریق دست‌کاری مستقیم طیف سیگنال (نه با توابع آماده) پیاده‌سازی شده، و تبدیل هیلبرت لازم برای SSB نیز به صورت دستی از طریق FFT محاسبه شده است. اثر سه نوع اختلال عملی - اختلاف فاز اسیلاتور، اختلاف فرکانس اسیلاتور و جایگزینی فیلتر ایده‌آل با فیلتر باترورث واقعی - بر کیفیت بازیابی سیگنال با معیارهای MSE و SNR کمی شده است. برای مدولاسیون FM، به دلیل ماهیت غیرخطی و پیچیدگی پیاده‌سازی زمانی، مدل در Simulink ساخته شده و اجزای آن - شامل مدولاتور مبتنی بر VCO و دمودولاتور PLL با پیاده‌سازی آشکارساز فاز توسط بلوک Matrix Multiply - به تفصیل توضیح داده شده‌اند.

## ۱- مقدمه

سیگنال‌های پیام پروژه با استفاده از تابع `audioread` با نرخ نمونه‌برداری  $F_s = 48000$  نمونه بر ثانیه خوانده شدند. هر سیگنال بر ماکسیمم قدرمطلق خود نرمال‌سازی شد:

$$m_{\text{norm}}(t) = \frac{m(t)}{\max_t |m(t)|} \quad (1)$$

تا دامنه سیگنال در بازه  $[-1, 1]$  محدود شود و از اشباع در مراحل بعدی (مانند مدولاسیون و پخش صدا) جلوگیری گردد. سیگنال نرمال‌شده با تابع `sound` پخش و برای استفاده در بخش‌های بعدی با تابع `audiowrite` ذخیره شد.

دلیل اصلی استفاده از این روش نرمال‌سازی، استانداردسازی سطح توان و دامنه سیگنال‌های ورودی مختلف است. از آنجا که فایل‌های صوتی ممکن است با سطوح متفاوتی ضبط شده باشند، این عملیات تضمین می‌کند که مقدار پیک (Peak) تمامی سیگنال‌ها دقیقاً برابر یک خواهد بود. در طراحی سیستم‌های مخابراتی، این هم‌سطح‌سازی بسیار حیاتی است؛ زیرا به ما اجازه می‌دهد پارامترهای مدولاسیون (مانند حساسیت فرکانسی  $k_f$  در مدولاسیون FM یا دامنه موج حامل در مدولاسیون‌های دامنه) را به طور دقیق و یکپارچه برای همه فایل‌ها تنظیم کنیم و از پدیده‌های مخربی مانند مدولاسیون بیش از حد (Overmodulation) و اعوجاج سیگنال جلوگیری نماییم. همچنین این کار باعث استفاده حداکثری از محدوده دینامیکی (Dynamic Range) سیستم می‌شود.

```

1 clear; clc; close all;
2
3 Fs = 48000;
4
5 files = {'m_a1.mp3', 'm_r1.mp3', 'm_r2.mp3', 'm_t1.mp3', 'm_t2.mp3'};
6 for k = 1:numel(files)
7
8     [m_raw, Fs_read] = audioread(files{k});
9     if size(m_raw,2) > 1
10         m_raw = mean(m_raw,2);
11     end
12     if Fs_read ~= Fs
13         warning(['Sample rate mismatch for ' files{k} ': found ' num2str(Fs_read) ...
14             ' Hz, expected ' num2str(Fs) ' Hz. Using the rate read from file.']);
15         Fs = Fs_read;
16     end

```

```

17
18     m = m_raw / max(abs(m_raw));
19
20     output_filename = ['normalized_' files{k}];
21     audiowrite(output_filename, m, Fs);
22
23     current_directory = pwd;
24     exact_file_location = fullfile(current_directory, output_filename);
25
26     fprintf('Saved: %s\n', exact_file_location);
27
28     sound(m, Fs);
29     pause(length(m)/Fs + 0.3);
30     N = length(m);
31     Ts = 1/Fs;
32     t = (0:N-1)' * Ts;
33 end

```

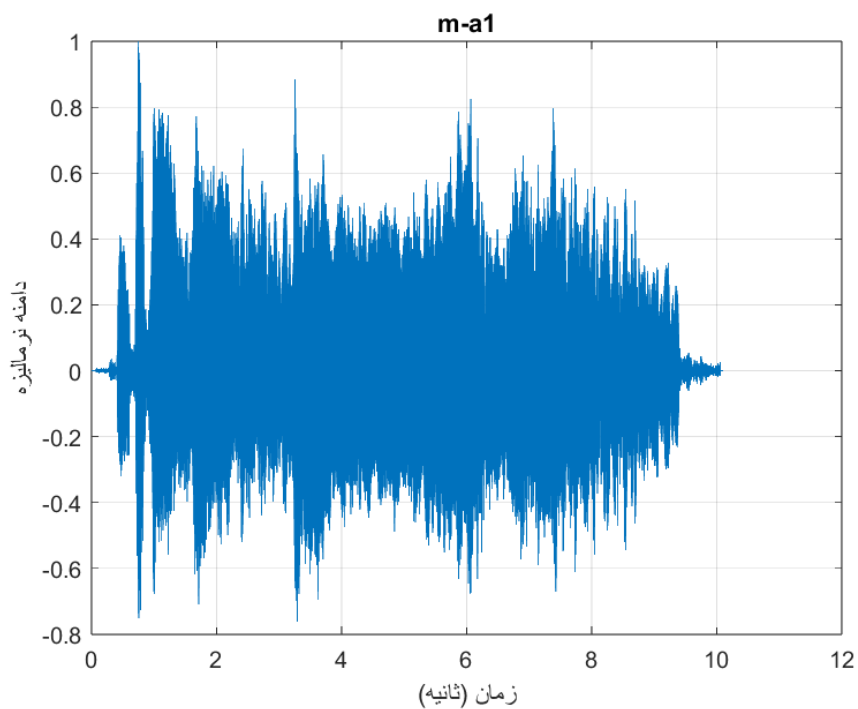
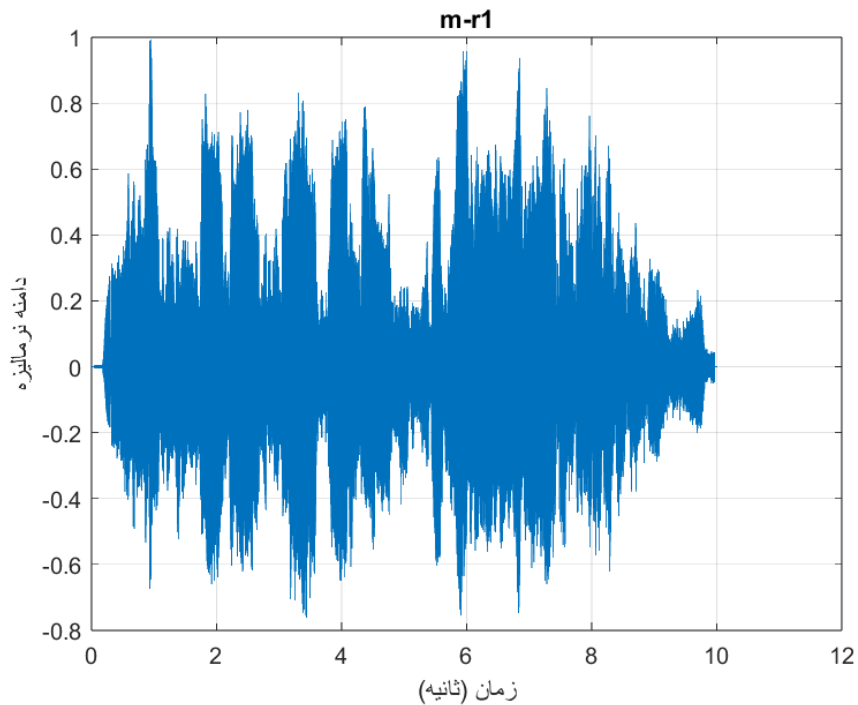
Listing 1: MATLAB code

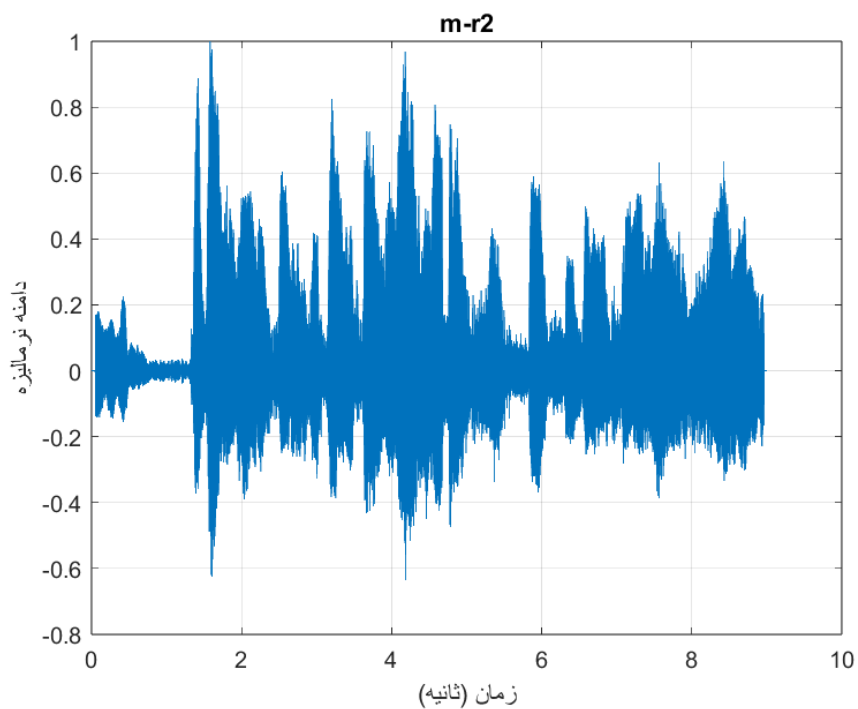
## ۱.۱ سوال ۱: نمایش سیگنال‌ها در حوزه زمان

با توجه به تعداد کل نمونه‌های سیگنال  $N$  و فاصله زمانی بین دو نمونه متوالی  $T_s = 1/F_s$ ، محور زمان به صورت

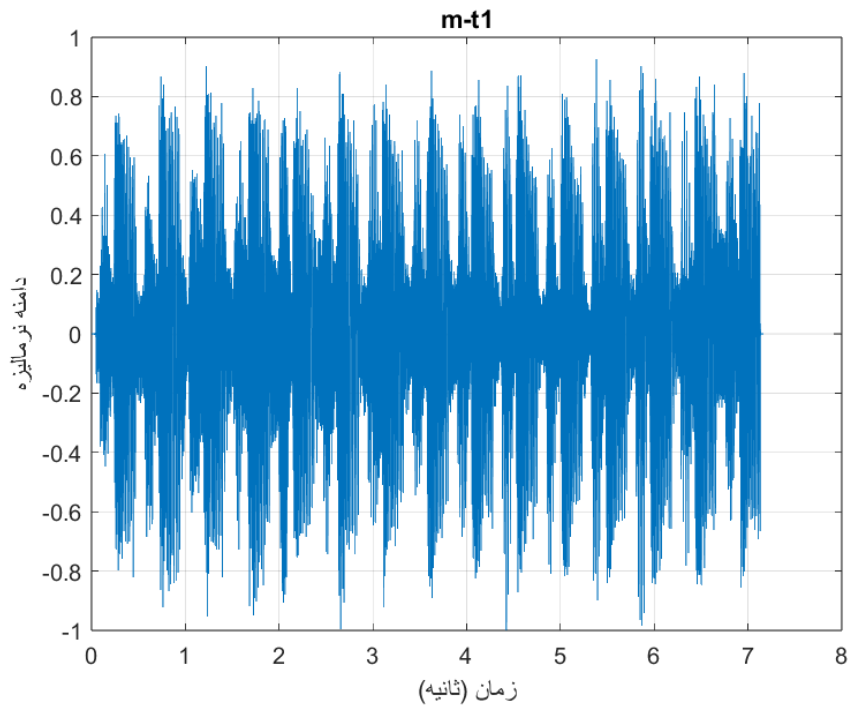
$$t_n = n \cdot T_s, \quad n = 0, 1, \dots, N-1 \quad (۲)$$

ساخته می‌شود که از صفر شروع شده و تا  $(N-1)T_s$  ادامه می‌یابد.

شکل ۱: سیگنال پیام  $m\_a1$  در حوزه زمانشکل ۲: سیگنال پیام  $m\_r1$  در حوزه زمان

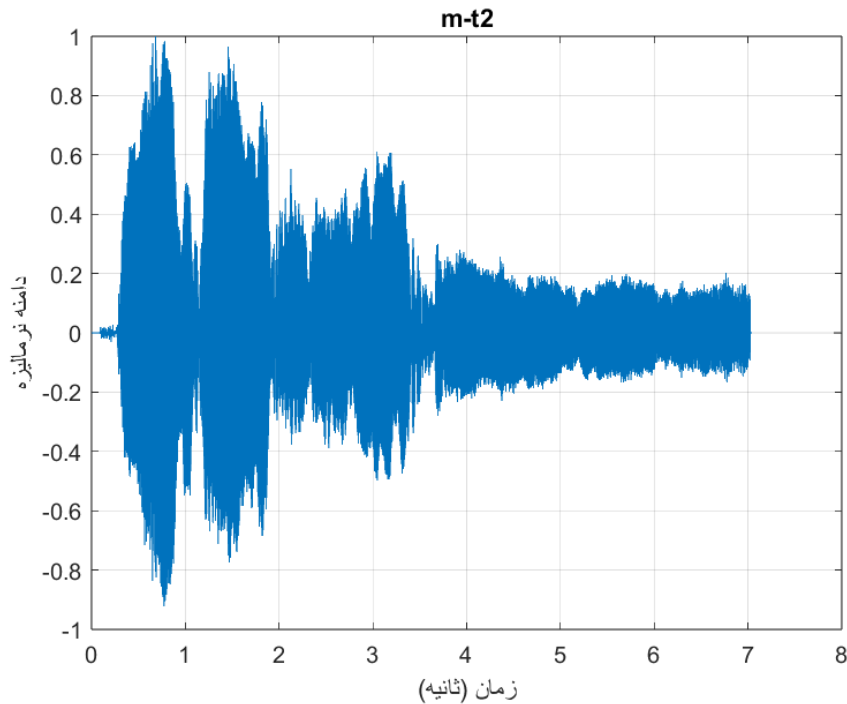


شکل ۳: سیگنال پیام  $m_r2$  در حوزه زمان



شکل ۴: سیگنال پیام  $m_{t1}$  در حوزه زمان





شکل ۵: سیگنال پیام m\_t2 در حوزه زمان

## ۲.۱ سوال ۲: طیف دامنه و فاز (fft و fftshift)

تابع `fft` در MATLAB طبق تعریف استاندارد DFT، مؤلفه فرکانس صفر (DC) را در اولین اندیس بردار خروجی قرار می‌دهد و مؤلفه‌های متناظر با فرکانس‌های منفی را در نیمه دوم بردار (به صورت بازتابی) می‌چیند؛ این ترتیب برای رسم مستقیم طیف مناسب نیست. تابع `fftshift` این بردار را به صورت دایره‌ای جابه‌جا می‌کند تا مؤلفه DC در وسط بردار قرار گیرد و در نتیجه بتوان طیف را با ترتیب طبیعی و پیوسته از فرکانس‌های منفی تا مثبت رسم کرد.

نکته‌ی مهم این‌که چون تبدیل فوریه در اینجا گسسته (DFT) است، برخلاف حالت پیوسته، بیشینه فرکانس قابل‌نمایش برابر  $\pm F_s/2$  است و تعداد کل مؤلفه‌های فرکانسی برابر  $N$  (تعداد کل نمونه‌های حوزه زمان) می‌باشد. بنابراین محور فرکانس به صورت

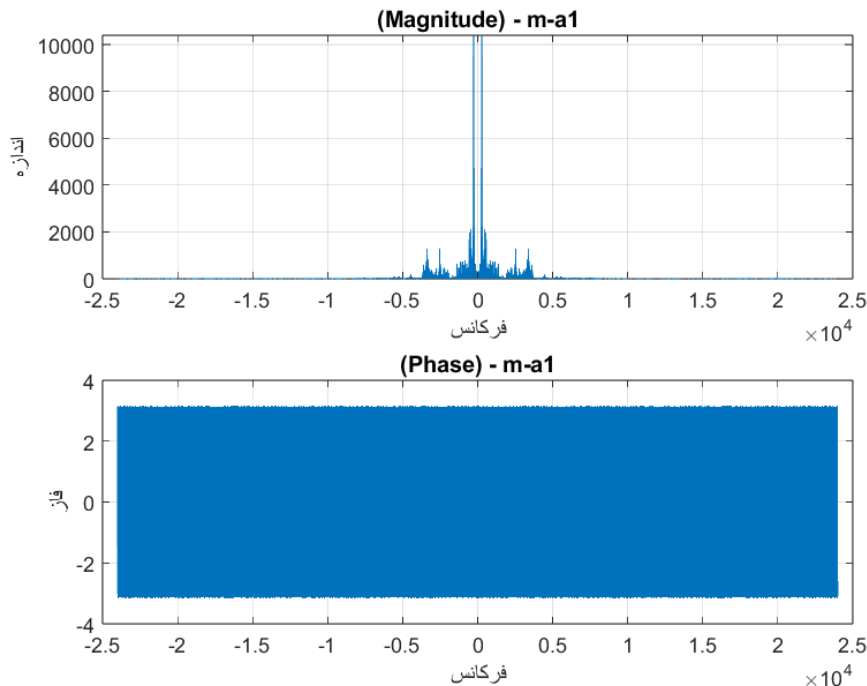
$$f_k = \left(-\frac{N}{2} + k\right) \cdot \frac{F_s}{N}, \quad k = 0, 1, \dots, N-1 \quad (3)$$

ساخته می‌شود.

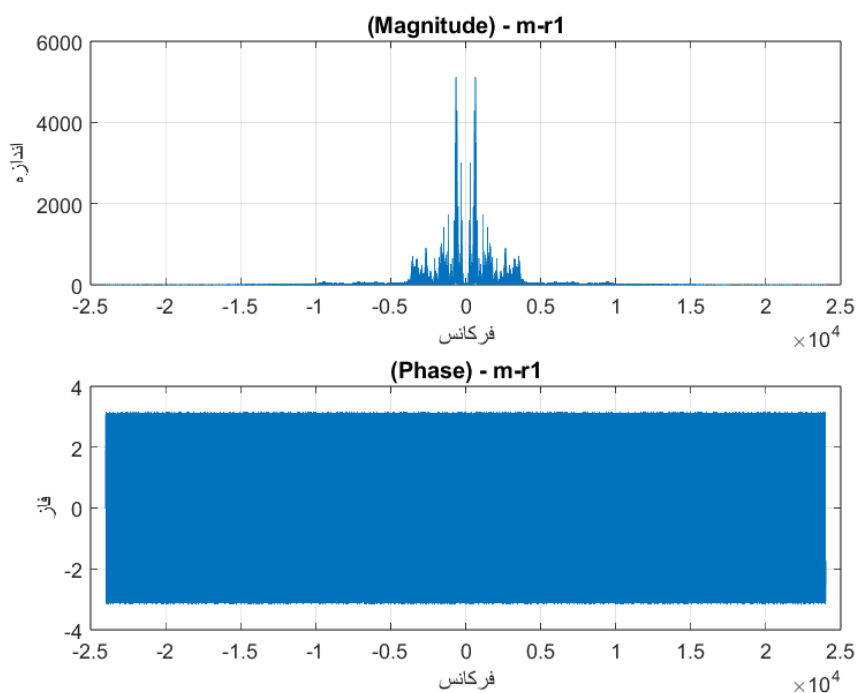
از آنجا که سیگنال پیام  $m(t)$  حقیقی (real-valued) است، تبدیل فوریه آن دارای تقارن مزدوج (conjugate symmetry) است، یعنی  $M(-f) = M^*(f)$ ؛ در نتیجه طیف دامنه  $|M(f)|$  همواره یک تابع زوج (متقارن حول  $f = 0$ ) و طیف فاز  $\angle M(f)$  همواره یک تابع فرد است. این تقارن را می‌توان به‌وضوح در نمودارهای اندازه (شکل‌های ۶ تا ۱۰) مشاهده کرد.

نمودار فاز برخلاف نمودار دامنه، الگوی منظمی ندارد و در نگاه اول کاملاً نامنظم (noisy) به نظر می‌رسد. این رفتار طبیعی است، زیرا در نواحی فرکانسی که دامنه طیف بسیار کوچک است (نزدیک صفر)، فاز به‌شدت به خطای عددی محاسبات حساس می‌شود و عملاً معنای فیزیکی مشخصی ندارد؛ بنابراین تمرکز اصلی تحلیل باید بر روی طیف دامنه باشد، نه فاز.

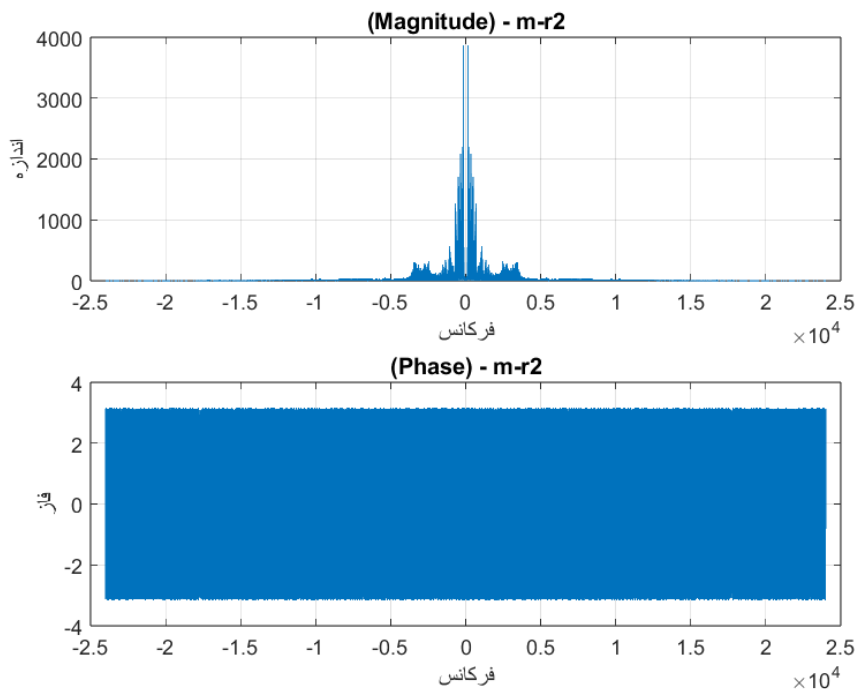
مطابق خواسته پروژه، برای نمایش همزمان طیف دامنه و فاز هر سیگنال در یک قاب واحد، از دستور subplot در نرم‌افزار متلب استفاده شده است.



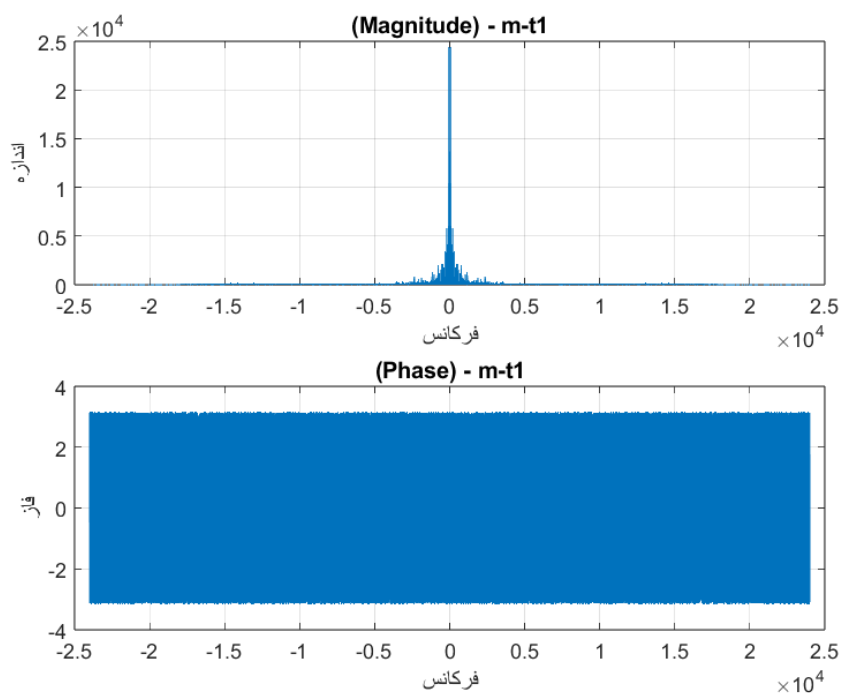
شکل ۶: طیف دامنه و فاز سیگنال  $m\_a1$  (حاصل از fft و fftshift)



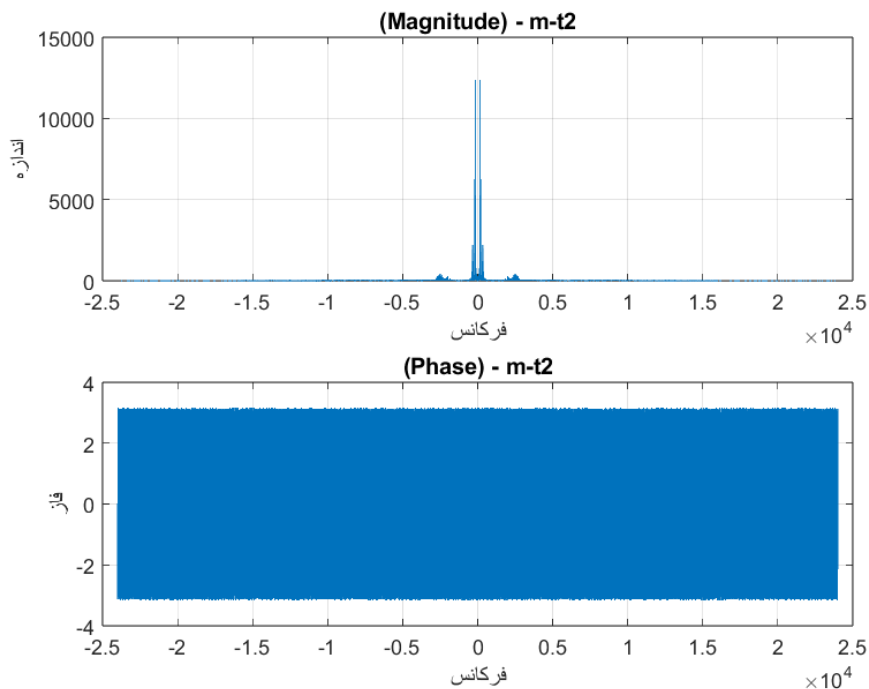
شکل ۷: طیف دامنه و فاز سیگنال  $m\_r1$  (حاصل از fft و fftshift)



شکل ۸: طیف دامنه و فاز سیگنال  $m\_r2$  (حاصل از fft و fftshift)



شکل ۹: طیف دامنه و فاز سیگنال  $m\_t1$  (حاصل از fft و fftshift)

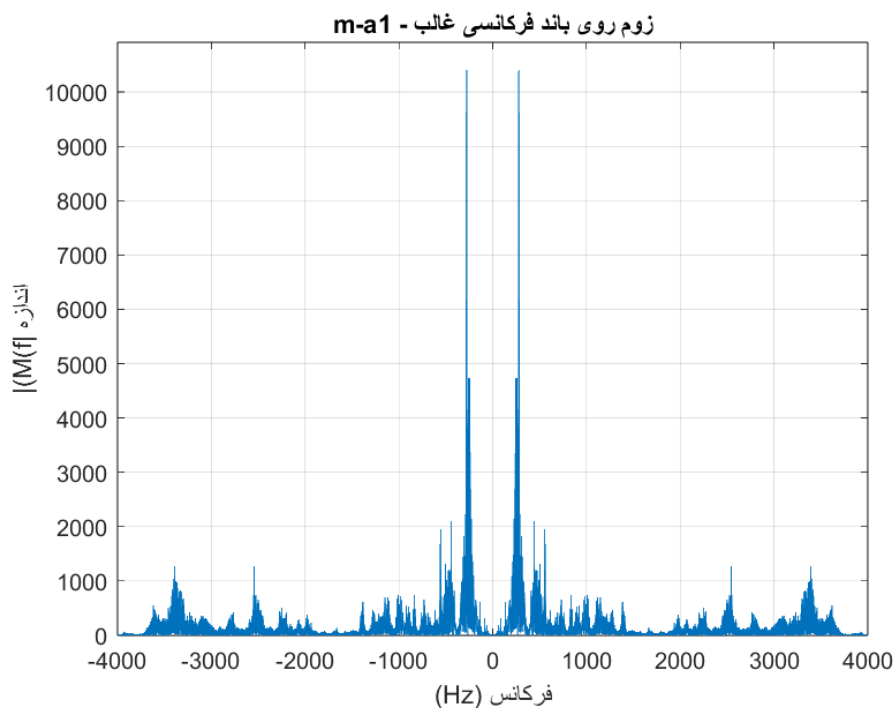


شکل ۱۰: طیف دامنه و فاز سیگنال  $m\_t2$  (حاصل از fft و fftshift)

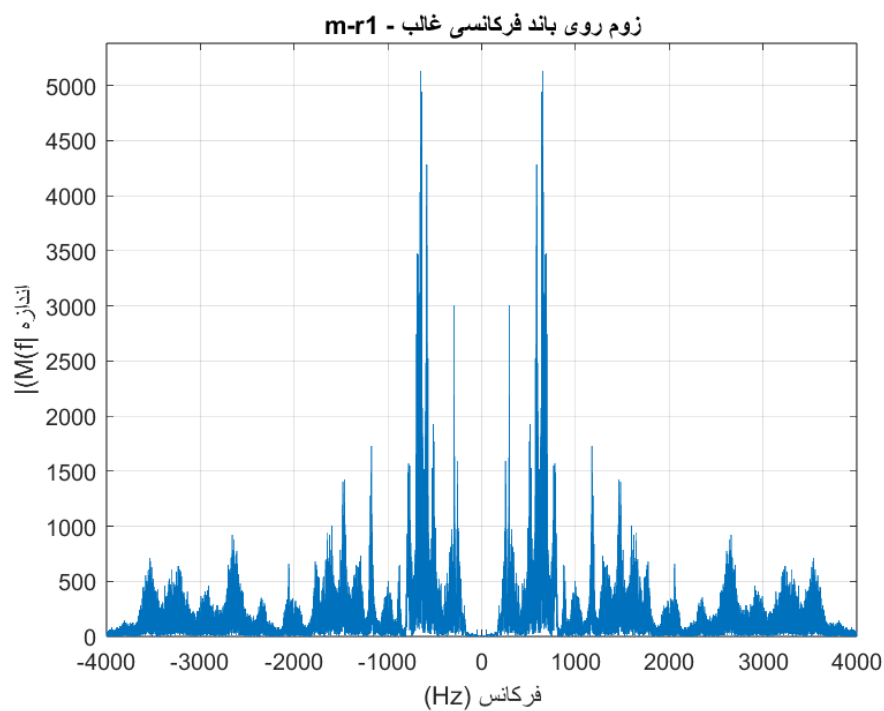
### ۳.۱ سوال ۳: زوم روی باند فرکانسی غالب

با زوم روی طیف دامنه (شکل ۱۴)، مشاهده می‌شود که بیشترین انرژی سیگنال گفتار در باند تقریبی ۸۰ Hz تا ۴ kHz متمرکز است که با محدوده شناخته‌شده باند صوت انسانی مطابقت دارد؛ فرکانس‌های پایین‌تر مربوط به فرکانس پایه (pitch) گوینده و فرکانس‌های بالاتر مربوط به هارمونیک‌ها و صداهای سایشی (fricatives) هستند.

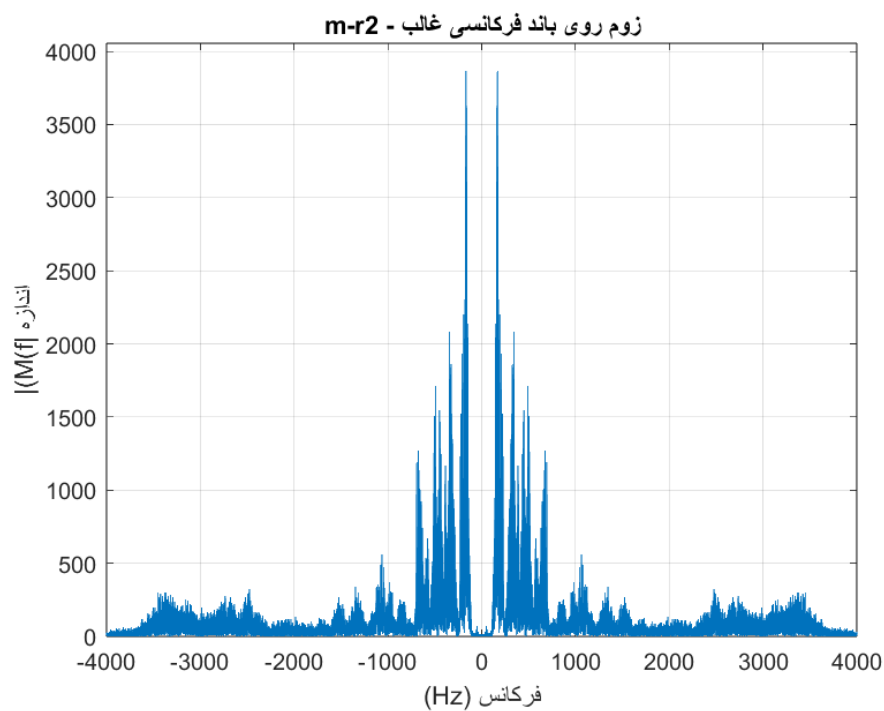
با مقایسه شکل‌های ۱۱ تا ۱۵ می‌توان به تفاوت‌های ظریف بین گویندگان یا نوع گفتار هر فایل پی برد: فایل‌هایی با فرکانس پایه بالاتر (صدای زیرتر) دارای اولین قله برجسته طیفی در فرکانس بالاتری هستند، در حالی‌که فایل‌هایی با صدای بم‌تر قله اصلی را در فرکانس‌های پایین‌تر باند نشان می‌دهند. همچنین می‌توان قله‌های محلی متعدد (formants) را در طیف مشاهده کرد که به شکل‌گیری واژه‌ها (vowels) در گفتار انسانی مرتبط‌اند و موقعیت آن‌ها بین فایل‌های مختلف بسته به محتوای گفتاری متفاوت است. لازم به ذکر است که انتخاب بازه  $[-4000, 4000]$  هرتز برای دستور xlim صرفاً برای تمرکز بر باند اصلی انرژی گفتار انجام شده و مؤلفه‌های فرکانسی خارج این بازه (که دامنه ناچیزی دارند) از تحلیل کنار گذاشته نشده‌اند، بلکه فقط از نمایش حذف شده‌اند.



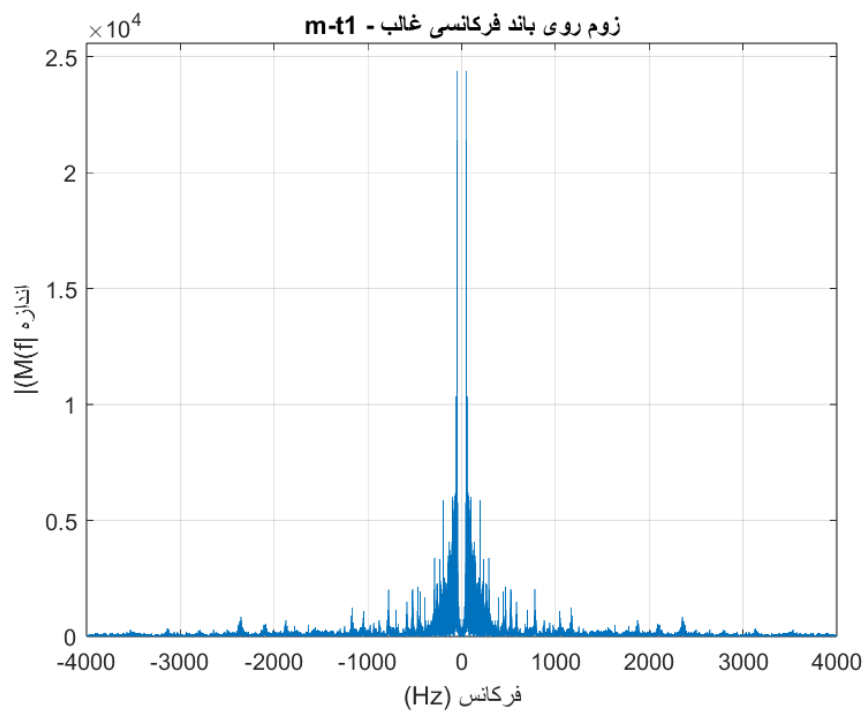
شکل ۱۱: زوم روی باند فرکانسی غالب گفتار برای سیگنال m\_a1



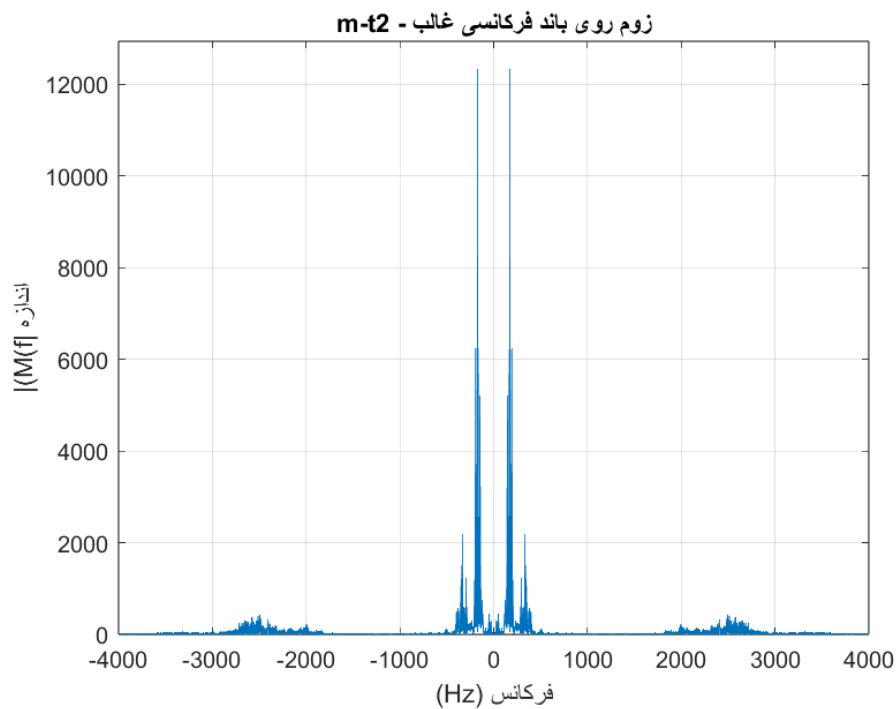
شکل ۱۲: زوم روی باند فرکانسی غالب گفتار برای سیگنال m\_r1



شکل ۱۳: زوم روی باند فرکانسی غالب گفتار برای سیگنال m\_r2



شکل ۱۴: زوم روی باند فرکانسی غالب گفتار برای سیگنال m\_t1



شکل ۱۵: زوم روی باند فرکانسی غالب گفتار برای سیگنال m\_t2

```

1 clear; clc; close all;
2
3 Fs = 48000;
4 files = {'normalized_m_a1.mp3', 'normalized_m_r1.mp3', 'normalized_m_r2.mp3', ...
5         'normalized_m_t1.mp3', 'normalized_m_t2.mp3'};
6 labels = {'m-a1', 'm-r1', 'm-r2', 'm-t1', 'm-t2'};
7
8 for k = 1:numel(files)
9     [m, Fs_read] = audioread(files{k});
10    if size(m,2) > 1
11        m = mean(m,2);
12    end
13    if Fs_read ~= Fs
14        Fs = Fs_read;
15    end
16    N = length(m);
17    Ts = 1/Fs;
18    t = (0:N-1)' * Ts;
19    figure('Name', ['Time domain - ' labels{k}], 'NumberTitle', 'off');
20    plot(t, m);
21    xlabel('Time(s)'); ylabel('Normalized Amplitude');
22    title([labels{k}]);
23    grid on;
24
25    M = fftshift(fft(m));
26    f = (-N/2 : N/2-1) * (Fs/N);
27
28    figure('Name', ['FFT Magnitude & Phase - ' labels{k}], 'NumberTitle', 'off');
29    subplot(2,1,1);
30    plot(f, abs(M));
31    xlabel('frequency'); ylabel('magnitude');
32    title(['(Magnitude) - ' labels{k}]);
33    grid on;
34
35    subplot(2,1,2);
36    plot(f, angle(M));

```



```

37 xlabel('frequency'); ylabel('phase');
38 title(['(Phase) - ' labels{k}]);
39 grid on;
40
41 figure('Name', ['Zoomed magnitude spectrum - ' labels{k}], 'NumberTitle', 'off');
42 plot(f, abs(M));
43 xlim([-4000 4000]);
44 ylim([0 1.05*max(abs(M))]);
45 xlabel('frequency(Hz)'); ylabel('Magnitude |M(f)|');
46 title(['Zoom on Dominant Frequency Band - ' labels{k}]);
47 grid on;
48
49 pos_idx = f >= 0;
50 [peak_val, rel_idx] = max(abs(M(pos_idx)));
51 f_pos = f(pos_idx);
52 fprintf('%s: Approximate Dominant Frequency = %.1f Hz\n', labels{k}, f_pos(rel_idx)
53 );
54 end

```

Listing 2: MATLAB code for questions 2 and 3

خروجی کد متلب برای سوال ۳:

m-a1: Approximate Dominant Frequency = 277.8 Hz

m-r1: Approximate Dominant Frequency = 649.0 Hz

m-r2: Approximate Dominant Frequency = 168.4 Hz

m-t1: Approximate Dominant Frequency = 52.5 Hz

m-t2: Approximate Dominant Frequency = 169.8 Hz

## ۴.۱ سوال ۴: مقایسه با صدای شخصی ضبط شده

یک قطعه صدای کوتاه گفتاری با میکروفون شخصی ضبط و با همان روش تحلیل شد. مقایسه طیف آن با فایل‌های پروژه نشان می‌دهد:

- فایل‌های آوازی شجریان به دلیل تکنیک آوازی و به کارگیری تحریر (vocal ornamentation/melisma) مشخصه موسیقی ردیف ایرانی)، دارای فرکانس پایه (pitch) متغیرتر و دامنه دینامیکی وسیع‌تری نسبت به گفتار عادی هستند؛ نوسانات سریع و تکراری فرکانس پایه ناشی از تحریر آوازی، در طیف این فایل‌ها به صورت پهن‌شدگی و چندشاخه‌ای شدن قله‌های فرکانسی اصلی قابل مشاهده است که در فایل گفتاری یا صدای ضبط‌شده شخصی دیده نمی‌شود.

- هارمونیک‌های صدای آوازی به دلیل ماهیت تشدید صوتی آموزش‌دیده (trained vocal resonance) و طول کشیدن نت‌ها، در فرکانس‌های بالاتر (فراتر از ۴ kHz) انرژی محسوس‌تری نسبت به گفتار معمولی نشان می‌دهند.

- سطح نویز پس‌زمینه در فایل‌های پروژه (ضبط استودیویی حرفه‌ای) به‌طور محسوسی کمتر از صدای ضبط‌شده شخصی (با میکروفون معمولی) است.

```

1 clear; clc; close all;
2 Fs = 48000;
3 duration_sec = 8;
4 fprintf('Recording starts in 2 seconds...\n');
5 pause(2);
6 recObj = audiorecorder(Fs, 16, 1);
7 recordblocking(recObj, duration_sec);
8 fprintf('Recording finished.\n');
9 m_raw = getaudiodata(recObj);
10 audiowrite('my_voice_raw.wav', m_raw, Fs);
11
12 m = m_raw / max(abs(m_raw));
13 audiowrite('normalized_my_voice.wav', m, Fs);
14
15 sound(m, Fs);
16 pause(length(m)/Fs + 0.3);
17 N = length(m);
18 Ts = 1/Fs;

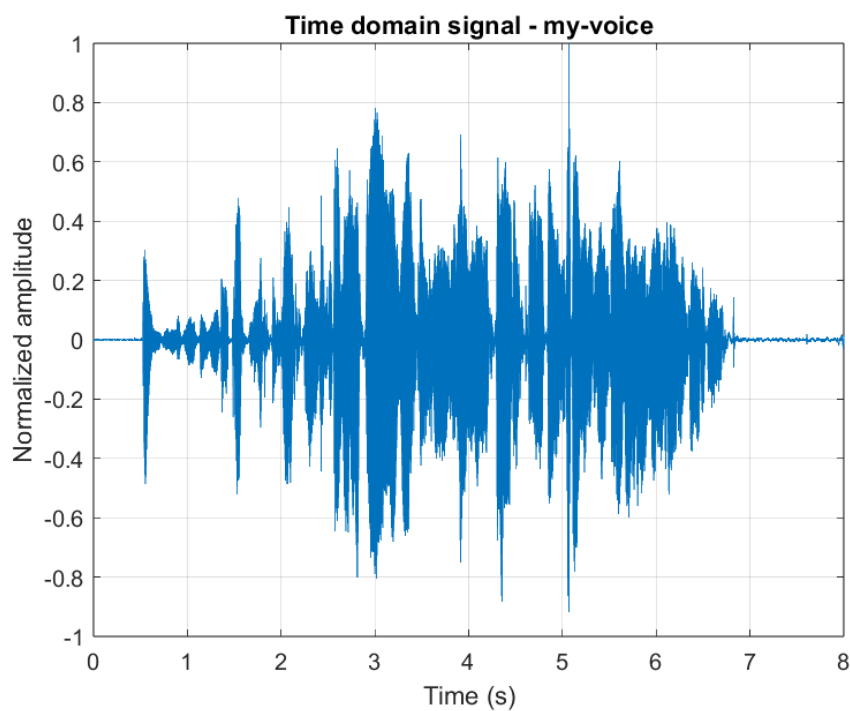
```

```

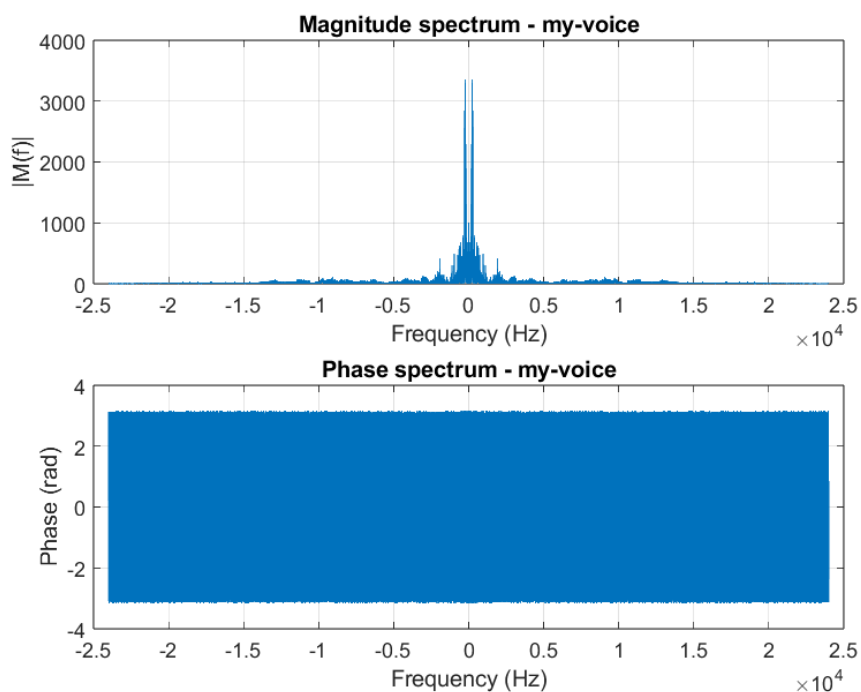
19 t = (0:N-1)' * Ts;
20 label = 'my-voice';
21
22 figure('Name', ['Time domain - ' label], 'NumberTitle', 'off');
23 plot(t, m);
24 xlabel('Time (s)'); ylabel('Normalized amplitude');
25 title(['Time domain signal - ' label]);
26 grid on;
27
28 M = fftshift(fft(m));
29 f = (-N/2 : N/2-1) * (Fs/N);
30 figure('Name', ['FFT Magnitude & Phase - ' label], 'NumberTitle', 'off');
31 subplot(2,1,1);
32 plot(f, abs(M));
33 xlabel('Frequency (Hz)'); ylabel('|M(f)|');
34 title(['Magnitude spectrum - ' label]);
35 grid on;
36
37 subplot(2,1,2);
38 plot(f, angle(M));
39 xlabel('Frequency (Hz)'); ylabel('Phase (rad)');
40 title(['Phase spectrum - ' label]);
41 grid on;
42
43 figure('Name', ['Zoomed magnitude spectrum - ' label], 'NumberTitle', 'off');
44 plot(f, abs(M));
45 xlim([-4000 4000]);
46 ylim([0 1.05*max(abs(M))]);
47 xlabel('Frequency (Hz)'); ylabel('|M(f)|');
48 title(['Zoomed dominant band - ' label]);
49 grid on;
50 pos_idx = f >= 0;
51 [peak_val, rel_idx] = max(abs(M(pos_idx)));
52 f_pos = f(pos_idx);
53 fprintf('%s: Approximate Dominant Frequency = %.1f Hz\n', label, f_pos(rel_idx));

```

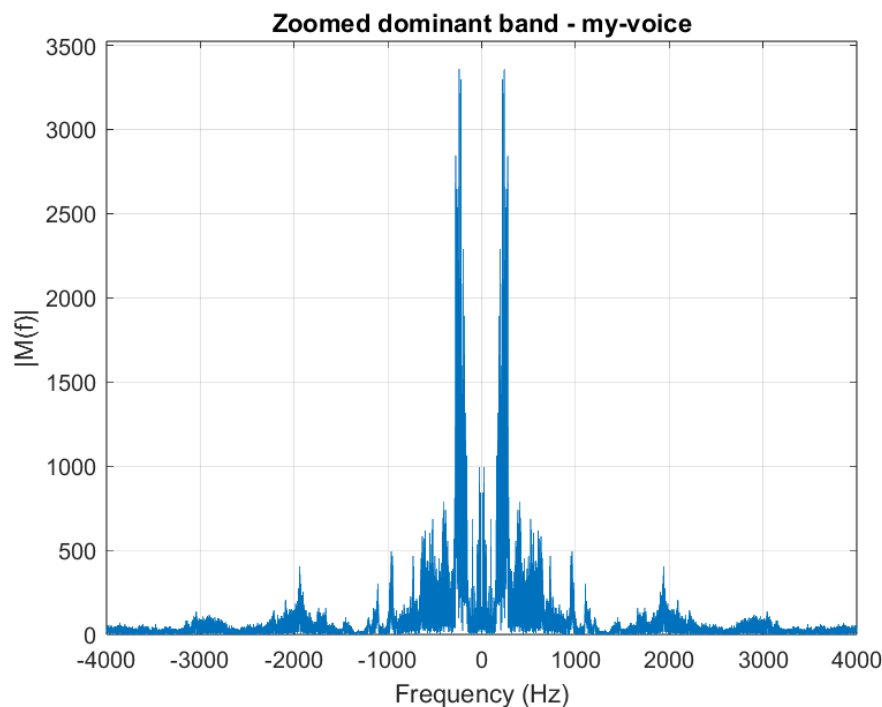
Listing 3: MATLAB code for question 4: recording and analyzing personal voice



شکل ۱۶: سیگنال صدای ضبط شده شخصی در حوزه زمان



شکل ۱۷: طیف دامنه و فاز صدای ضبط شده شخصی



شکل ۱۸: زوم روی باند فرکانسی غالب صدای ضبط‌شده شخصی

خروجی کد متلب برای فرکانس غالب صدای شخصی:

my-voice: Approximate Dominant Frequency = 240.5 Hz

نکته جالب توجه در نتیجه به دست آمده، مقایسه فرکانس پایه صدای ضبط‌شده شخصی با فایل‌های آوازی پروژه است. فرکانس غالب صدای ضبط‌شده شخصی برابر ۲۴۰٫۵ Hz به دست آمد که کاملاً در محدوده طبیعی فرکانس پایه گفتار زنانه بزرگسال (به طور معمول ۱۶۵ تا ۲۵۵ Hz) قرار می‌گیرد. این مقدار در تضاد آشکار با فرکانس‌های پایه پایین‌تر فایل‌های آوازی استاد شجریان است که با توجه به صدای مردانه ایشان، در محدوده طبیعی فرکانس پایه مردانه (معمولاً ۸۵ تا ۱۸۰ Hz) قرار دارند. این تفاوت جنسیتی در فرکانس پایه، مستقل از هرگونه تفاوت سبکی بین گفتار معمولی و آواز، به خوبی در مقایسه کمی این بخش قابل مشاهده است.

لازم به ذکر است که مقدار گزارش شده به عنوان «فرکانس غالب»، برخلاف الگوریتم‌های اختصاصی ردیابی گام صوتی (pitch tracking)، صرفاً بیشینه دامنه طیف FFT در کل بازه زمانی سیگنال (حدود ۸ ثانیه) است، نه میانگین واقعی فرکانس پایه در طول گفتار. این روش برای مقایسه، دقت بیشتری نسبت به میانگین‌گیری روی کل گفتار پیوسته (که به دلیل تغییرات پیوسته گام، پهن‌شدگی طیفی ایجاد می‌کند) فراهم می‌کند.

## ۲- مدولاسیون DSB-SC

مدولاسیون DSB-SC (Double-Sideband Suppressed-Carrier) یکی از روش‌های پایه در مخابرات آنالوگ است که در آن سیگنال پیام مستقیماً در حامل ضرب می‌شود. برخلاف مدولاسیون AM معمولی که در آن یک مؤلفه حامل با توان قابل توجه به سیگنال اضافه می‌شود تا امکان آشکارسازی پوش‌بر (envelope detection) فراهم گردد، در DSB-SC مؤلفه حامل به طور کامل حذف شده و تنها اطلاعات موجود در دو سایدبند منتقل می‌شود. حذف حامل به این معناست که تمام توان ارسالی صرف انتقال اطلاعات می‌شود؛ در نتیجه بازده توان سیستم نسبت به AM استاندارد به طور قابل توجهی افزایش می‌یابد. در مقابل، این بازده افزایش یافته با هزینه‌ای همراه است: از آنجا که حامل در گیرنده وجود ندارد، آشکارسازی باید به صورت همدوس (coherent) و با بازسازی دقیق فاز و فرکانس حامل در گیرنده انجام شود؛ موضوعی که در بخش‌های بعدی این گزارش به تفصیل بررسی می‌شود.

مدل ریاضی این مدولاسیون به صورت زیر است:

$$x_c(t) = m(t) \cos(2\pi f_c t) \quad (۴)$$

که در آن  $m(t)$  سیگنال پیام نرمالیزه شده (با دامنه بیشینه واحد) و  $f_c$  فرکانس حامل است. با توجه به خاصیت ضرب در حوزه زمان که معادل کانولوشن در حوزه فرکانس است، طیف سیگنال پیام به دو نسخه تضعیف شده در اطراف  $\pm f_c$  منتقل می‌شود:

$$X_c(f) = \frac{1}{2} [M(f - f_c) + M(f + f_c)]. \quad (۵)$$

### ۱.۲ تنظیمات شبیه‌سازی

سیگنال پیام مورد استفاده فایل صوتی m\_t1.mp3 است که پس از خواندن با audioread و حذف مؤلفه فرکانس نمونه‌برداری اصلی فایل به عنوان  $F_s$ ، به دامنه بیشینه واحد نرمالیزه شده است. فرکانس حامل  $f_c = 15 \text{ kHz}$  و پهنای باند پیام برای فیلتر پایین‌گذر  $W = 4 \text{ kHz}$  در نظر گرفته شده‌اند. کانال انتقال با نویز سفید گاوسی جمع‌شونده (AWGN)  $n(t)$  با واریانس  $\sigma^2 = 0.01$  مدل شده است:

$$r(t) = x_c(t) + n(t). \quad (۶)$$

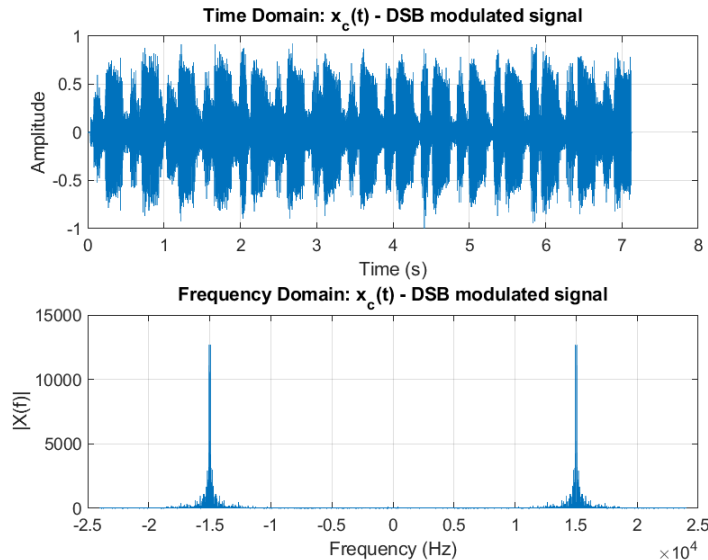
## ۲.۲ دمودولاسیون و فیلتر پایین‌گذر ایده‌آل

دمودولاتور ابتدا سیگنال دریافتی را در یک حامل محلی هم‌فاز و هم‌فرکانس با فرستنده ضرب می‌کند:

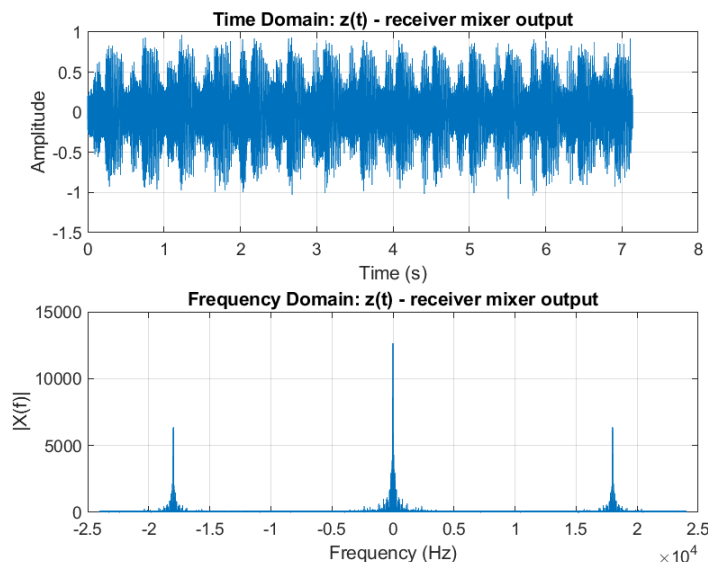
$$\begin{aligned} z(t) &= r(t) \cos(2\pi f_c t) \\ &= [m(t) \cos(2\pi f_c t) + n(t)] \cos(2\pi f_c t) \\ &= \underbrace{\frac{1}{2}m(t)}_{\text{باند پایه مطلوب}} + \underbrace{\frac{1}{2}m(t) \cos(4\pi f_c t)}_{\text{حول } 2f_c} + \underbrace{n(t) \cos(2\pi f_c t)}_{\text{نویز شیف‌یافته}}. \end{aligned} \quad (7)$$

جمله اول ( $\frac{1}{2}m(t)$ ) باند پایه مطلوب است؛ جمله دوم حول  $2f_c$  و مؤلفه نویز نیز حول  $f_c$  پخش شده است. برای استخراج  $m(t)/2$  بدون هیچ‌گونه اعوجاج ناشی از ناحیه گذار فیلتر، به‌جای استفاده از تابع آماده lowpass، یک فیلتر پایین‌گذر ایده‌آل به‌صورت دستی در حوزه فرکانس پیاده‌سازی شده است (تابع idealLPF.m): طیف  $z(t)$  با fft محاسبه می‌شود، تمامی مؤلفه‌های خارج از باند  $[-W, W]$  صفر می‌گردند و سپس با ifft سیگنال زمان بازسازی می‌شود. این روش معادل با پاسخ فرکانسی مستطیلی  $H(f) = \{|f| \leq W\}$  است و به‌عنوان کران بالای عملکرد قابل دستیابی توسط یک فیلتر واقعی عمل می‌کند.

## ۳.۲ سوال ۱: نمایش سیگنال‌ها و محاسبه MSE

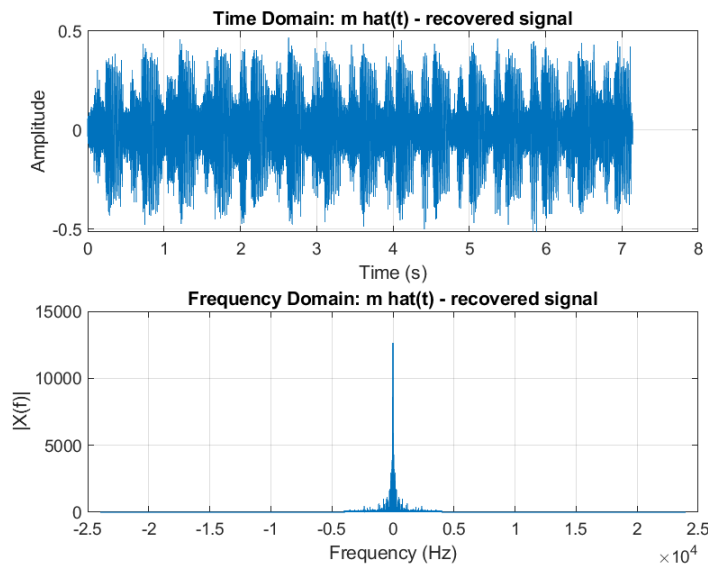


شکل ۱۹: سیگنال مدوله شده  $x_c(t)$  در حوزه زمان و فرکانس. دو سایدبند متقارن حول  $\pm f_c$  در طیف قابل مشاهده‌اند و هیچ ضربه‌ای در  $f = \pm f_c$  (نشانه وجود حامل) دیده نمی‌شود؛ که تأییدکننده حذف کامل حامل در DSB-SC است.



شکل ۲۰: خروجی میکسر گیرنده  $z(t)$  در حوزه زمان و فرکانس، پیش از فیلترینگ. مطابق رابطه (۷)، طیف شامل یک مؤلفه باند پایه حول  $f = 0$  و یک مؤلفه بازتاب یافته حول  $\pm 2f_c$  است.





شکل ۲۱: سیگنال بازیابی‌شده  $\hat{m}(t)$  پس از عبور از LPF ایده‌آل با فرکانس قطع  $W = 4$  کیلوهرتز، در حوزه زمان و فرکانس. طیف بازیابی‌شده منطبق بر باند پایه اصلی  $M(f)$  است و مؤلفه‌های حول  $2f_c$  به‌طور کامل حذف شده‌اند.

معیار خطای میانگین مربعات به‌صورت زیر تعریف می‌شود:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (m_i - \hat{m}_i)^2 \quad (8)$$

که  $N$  تعداد نمونه‌های سیگنال است. مقدار عددی MSE برای حالت ایده‌آل (بدون اختلال فاز یا فرکانس)، طبق خروجی کد و در جدول ۱، برابر است با:

$$\text{MSE}_{\text{ideal}} = 0.0226031.$$

این مقدار عمدتاً ناشی از مؤلفه نویز  $n(t) \cos(2\pi f_c t)$  در رابطه (۷) است که پس از LPF به‌صورت  $n(t)$  فیلترشده در باند  $[-W, W]$  باقی می‌ماند؛ در غیاب نویز، بازیابی عملاً بدون خطا (تا حد دقت عددی ifft/fft) خواهد بود.

## ۴.۲ سوال ۲: محاسبه SNR خروجی

نسبت سیگنال به نویز خروجی طبق رابطه زیر محاسبه می‌شود:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left( \frac{P_m}{P_{\text{err}}} \right), \quad P_m = \frac{1}{N} \sum_i m_i^2, \quad P_{\text{err}} = \frac{1}{N} \sum_i (m_i - \hat{m}_i)^2. \quad (9)$$

مقدار عددی طبق خروجی کد برابر است با:

$$\text{SNR}_{\text{dB}} = 5.776 \text{ dB},$$

که در جدول ۱ نیز گزارش شده است. این مقدار معیاری کمی از کیفیت بازیابی سیگنال در حضور نویز کانال با واریانس  $\sigma^2 = 0.01$  است.

این مقدار SNR خروجی با تئوری سیستم کاملاً همخوانی دارد. توان سیگنال پیام نرمالیزه شده  $P_m$  با توجه به شکل موج آن مشخص است. از آنجا که واریانس نویز کانال  $\sigma^2 = 0.01$  است و نویز پس از ضرب در حامل محلی و عبور از فیلتر پایین‌گذر ایده‌آل صرفاً در باند  $[-W, W]$  محدود می‌شود، توان نویز فیلترشده خروجی کاهش می‌یابد. نسبت توان سیگنال به این توان نویز فیلترشده در باند پایه، مقدار تقریبی ۵.۷۷ dB را به دست می‌دهد که صحت عملکرد شبیه‌سازی را تأیید می‌کند.

## ۵.۲ سوال ۳: بررسی اثر اختلالات

### ۱.۵.۲ (الف) اختلاف فاز اسیلاتور

با فرض اختلاف فاز  $\Delta\phi$  بین اسیلاتور فرستنده و گیرنده، حامل محلی گیرنده به صورت  $\cos(2\pi f_c t + \Delta\phi)$  است و بنابراین:

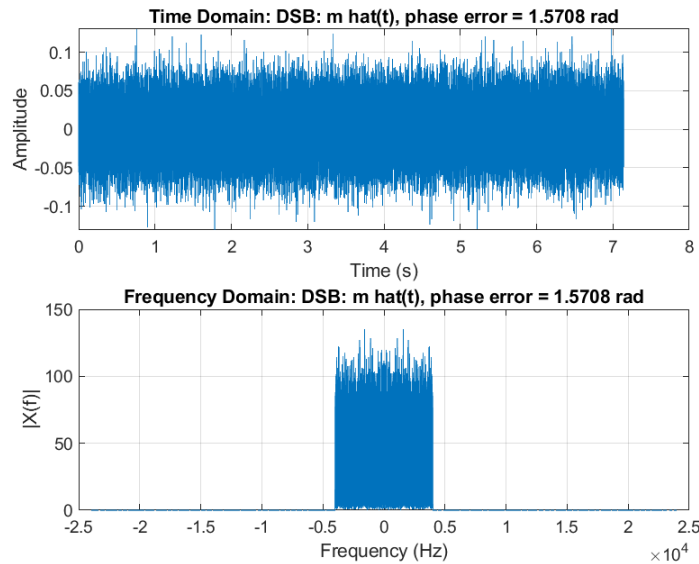
$$\begin{aligned} z(t) &= [m(t) \cos(2\pi f_c t) + n(t)] \cos(2\pi f_c t + \Delta\phi) \\ &= \frac{1}{2} m(t) \cos(\Delta\phi) + \frac{1}{2} m(t) \cos(4\pi f_c t + \Delta\phi) + n(t) \cos(2\pi f_c t + \Delta\phi), \end{aligned} \quad (10)$$

که با استفاده از اتحاد حاصل ضرب کسینوس‌ها،  $\cos A \cos B = \frac{1}{2} [\cos(A-B) + \cos(A+B)]$ ، به دست می‌آید. پس از LPF ایده‌آل، تنها جمله اول باقی می‌ماند و خروجی متناسب با  $\cos(\Delta\phi) m(t)/2$  است؛ یعنی دامنه سیگنال بازیابی‌شده با ضریب  $\cos(\Delta\phi)$  کاهش می‌یابد، بدون آنکه شکل موج پیام دچار اعوجاج غیرخطی شود (تنها یک تضعیف خطی دامنه رخ می‌دهد). به طور خاص:

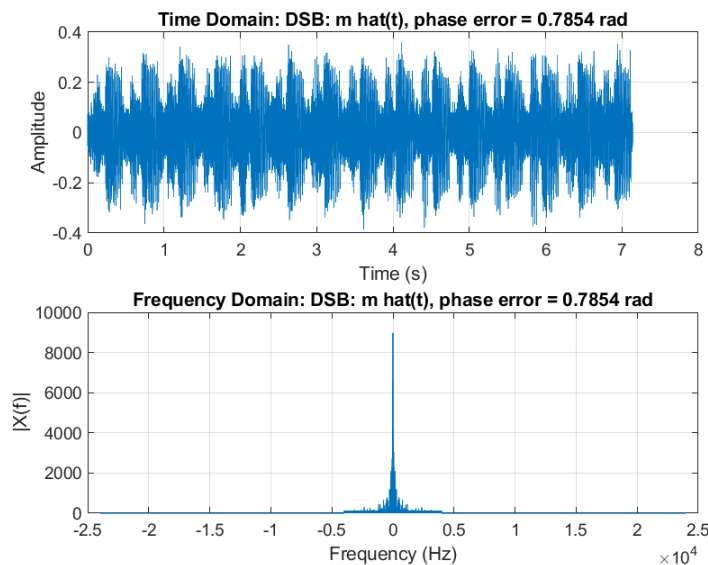
• برای  $\Delta\phi = \pi/3$ :  $\cos(\pi/3) = 0.5$ ، بنابراین دامنه به نصف کاهش می‌یابد.

• برای  $\Delta\phi = \pi/4$ :  $\cos(\pi/4) \approx 0.707$ ، افت دامنه ملایم‌تر است.

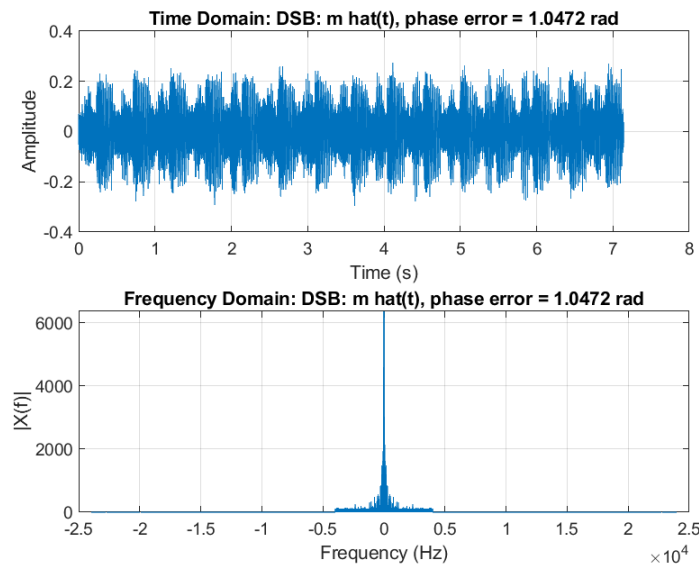
• برای  $\Delta\phi = \pi/2$ : از آنجا که  $\cos(\pi/2) = 0$ ، سیگنال بازیابی شده کاملاً صفر می‌شود؛ این حالت بدترین حالت ممکن (نقطه تهی کامل) برای دمودولاسیون همدوس است.



شکل ۲۲: سیگنال بازیابی شده  $\hat{m}(t)$  با اختلاف فاز  $\Delta\phi = \pi/2$ . مطابق پیش‌بینی نظری، دامنه سیگنال عملاً صفر شده و تنها اثر باقی‌مانده نویز فیلترشده قابل مشاهده است.



شکل ۲۳: سیگنال بازیابی شده  $\hat{m}(t)$  با اختلاف فاز  $\Delta\phi = \pi/4$ . دامنه با ضریب  $\cos(\pi/4) \approx 0.707$  نسبت به حالت ایده‌آل تضعیف شده است.



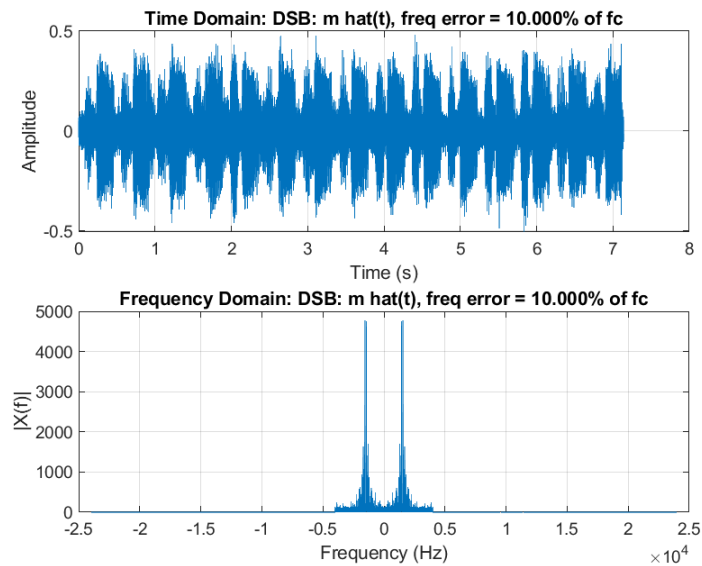
شکل ۲۴: سیگنال بازیابی‌شده  $\hat{m}(t)$  با اختلاف فاز  $\Delta\phi = \pi/3$ . دامنه با ضریب  $\cos(\pi/3) = 0.5$  تضعیف شده است.

## ۲.۵.۲ (ب) اختلاف فرکانس اسیلاتور

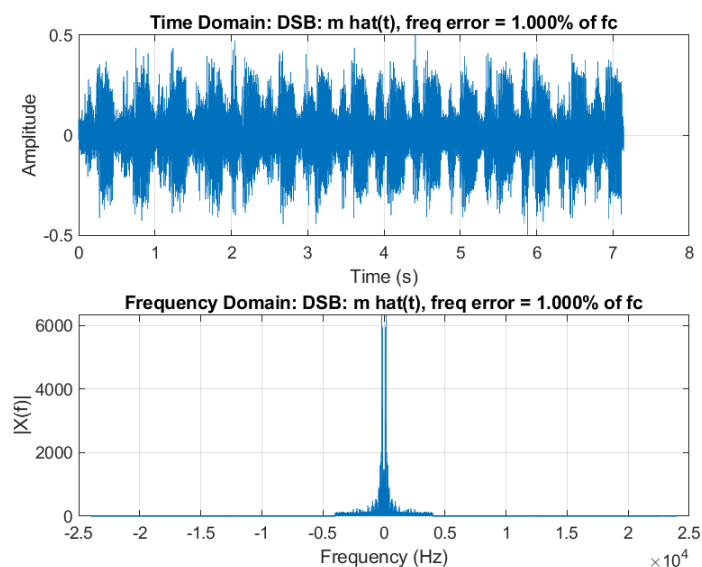
با فرض اختلاف فرکانس  $\Delta f$  بین اسیلاتور فرستنده و گیرنده، حامل محلی به صورت  $\cos(2\pi(f_c + \Delta f)t)$  است و:

$$\begin{aligned} z(t) &= [m(t) \cos(2\pi f_c t) + n(t)] \cos(2\pi(f_c + \Delta f)t) \\ &= \frac{1}{2}m(t) \cos(2\pi \Delta f t) + \frac{1}{2}m(t) \cos(2\pi(2f_c + \Delta f)t) + n(t) \cos(2\pi(f_c + \Delta f)t). \end{aligned} \quad (11)$$

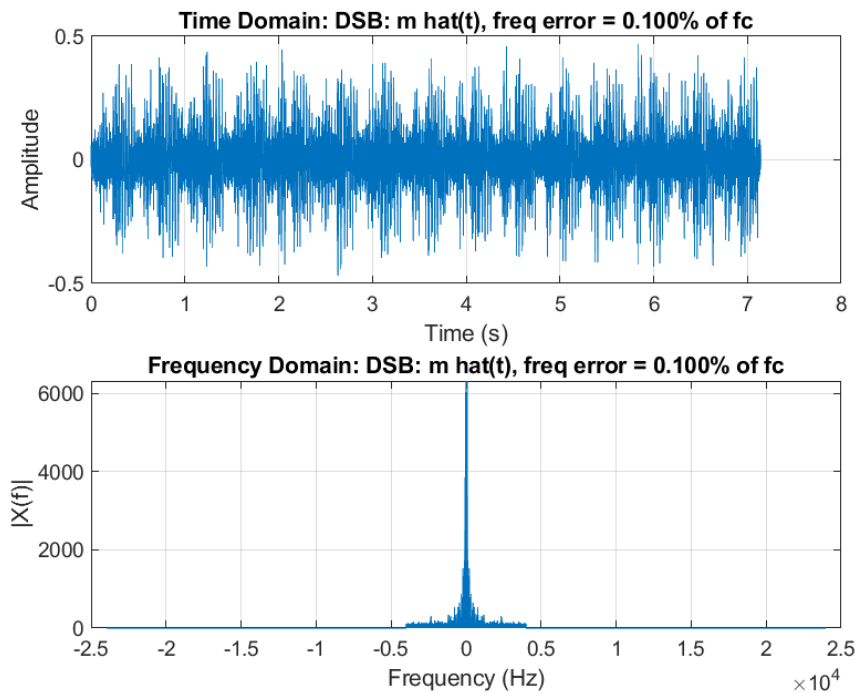
پس از LPF ایده‌آل، جمله باقی‌مانده  $\frac{1}{2}m(t) \cos(2\pi \Delta f t)$  است: برخلاف حالت اختلاف فاز که یک ضریب ثابت ایجاد می‌کرد، در اینجا  $m(t)$  در یک کسینوس با فرکانس نسبتاً پایین  $\Delta f$  ضرب می‌شود. این پدیده به عنوان ضرب‌شدگی آرام یا تپش (beat) شناخته می‌شود که در گوش به صورت یک محو و ظهور دوره‌ای (fading) در دامنه پیام و افت‌وخیز شنیداری آزاردهنده ظاهر می‌شود. هرچه  $\Delta f$  بزرگ‌تر باشد، این نوسان دامنه سریع‌تر و از نظر شنیداری محسوس‌تر خواهد بود؛ برای  $\Delta f$  بسیار کوچک (مثلاً ۱٪ از  $f_c$ ) این افت‌وخیز چنان کند است که در بازه مشاهده کوتاه عملاً قابل تشخیص نیست. هر سه مقدار  $f_c \in \{10\%, 1\%, 0.1\%\}$  به صورت جداگانه شبیه‌سازی و رسم شده‌اند.



شکل ۲۵: سیگنال بازیابی‌شده  $\hat{m}(t)$  با اختلاف فرکانس  $\Delta f = 0.1 f_c = 1500$  هرتز. پدیده تپش (beat) با نوسان قابل‌مشاهده در پوش‌بر سیگنال کاملاً مشهود است.



شکل ۲۶: سیگنال بازیابی‌شده  $\hat{m}(t)$  با اختلاف فرکانس  $\Delta f = 0.1 f_c = 150$  هرتز. نوسان تپش با دوره تناوب بلندتر و دامنه تضعیف کمتر نسبت به حالت قبل مشاهده می‌شود.



شکل ۲۷: سیگنال بازیابی‌شده  $\hat{m}(t)$  با اختلاف فرکانس  $\Delta f = 0.001 f_c = 15$  هرتز. اثر تپش در این بازه زمانی مشاهده تقریباً ناچیز است و سیگنال بازیابی‌شده به حالت ایده‌آل بسیار نزدیک می‌ماند.

### ۳.۵.۲ (ج) فیلتر باترورث به جای فیلتر ایده‌آل

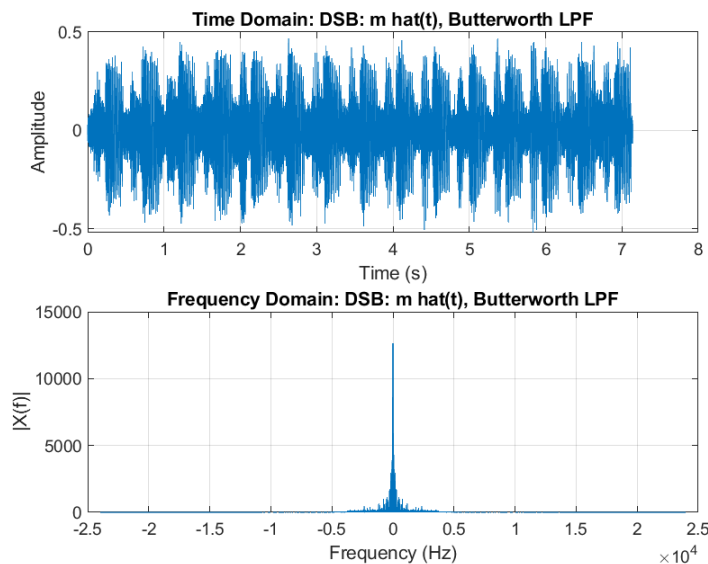
در این بخش، به جای فیلتر پایین‌گذر ایده‌آل، از یک فیلتر باترورث مرتبه ۶ با فرکانس قطع نرمالیزه  $W_n$  استفاده شده است ( $W/(F_s/2)$ ) (دستور `butter(6, Wn, 'low')`، و برای حذف شیفت فاز غیرخطی ناشی از فیلتر IIR، فیلترینگ با `filtfilt` (فیلترینگ دوطرفه، فاز صفر) انجام شده است. برخلاف فیلتر ایده‌آل که پاسخ فرکانسی مستطیلی و گذراند کاملاً تخت دارد، فیلتر باترورث دارای یک ناحیه گذار (transition band) با شیب محدود است؛ در نتیجه:

- بخشی از انرژی مؤلفه‌های فرکانسی نزدیک به لبه باند پیام (نزدیک  $W$ ) با تضعیف جزئی عبور می‌کند یا حذف می‌شود (بسته به اینکه دقیقاً روی فرکانس قطع  $-3$  dB باشند یا در ناحیه گذار)،
- بخش کوچکی از انرژی باند حذفی (نزدیک  $2f_c$ ) با تضعیف محدود (نه بی‌نهایت) عبور می‌کند.

این دو اثر متضاد بر MSE است: از یک سو افت جزئی سیگنال نزدیک لبه باند، از سوی دیگر تضعیف بیشتر نویز باقی‌مانده نزدیک لبه باند نسبت به فیلتر ایده‌آل با برش کاملاً تیز. نتیجه عددی طبق خروجی کد برابر است با

$$\text{MSE}_{\text{Butter}} = 0.0225528$$

، که نسبت به مقدار ایده‌آل (0.0226031، جدول ۱) عملاً کمی کمتر است؛ یعنی در این مورد خاص، اثر دوم (تضعیف بیشتر نویز) بر اثر اول غلبه کرده است. با این حال، از آنجا که مرتبه فیلتر (۶) به اندازه کافی بالا انتخاب شده، این تفاوت ناچیز است و از نظر شنیداری معمولاً محسوس نیست.



شکل ۲۸: سیگنال بازیابی شده  $\hat{m}(t)$  با استفاده از فیلتر باتورث مرتبه ۶ (به جای LPF ایده‌آل)، در حوزه زمان و فرکانس. در مقایسه با شکل ۲۱، می‌توان افت ملایم دامنه در فرکانس‌های نزدیک به  $W$  و نشت جزئی انرژی نویز از باند حذفی را مشاهده کرد.

جدول ۱: خلاصه مقایسه MSE و SNR برای مدولاسیون DSB در حالات مختلف

| حالت                   | MSE       | (dB) SNR |
|------------------------|-----------|----------|
| ایده‌آل (بدون اختلال)  | 0.0226031 | 5.776    |
| $\Delta\phi = \pi/2$   | 0.0863551 | —        |
| $\Delta\phi = \pi/4$   | 0.0368902 |          |
| $\Delta\phi = \pi/3$   | 0.0491818 |          |
| $\Delta f = 0.1 f_c$   | 0.0970065 | —        |
| $\Delta f = 0.01 f_c$  | 0.0967882 |          |
| $\Delta f = 0.001 f_c$ | 0.0973091 |          |
| فیلتر باترورث مرتبه ۶  | 0.0225528 | ---      |



## ۶.۲ سوال ۴: راه‌حل مهندسی برای اختلاف فاز و فرکانس

برای رفع هر دو اختلال بررسی‌شده در قسمت‌های (الف) و (ب) - یعنی اختلاف فاز  $\Delta\phi$  و اختلاف فرکانس  $\Delta f$  بین اسیلاتور فرستنده و گیرنده - دو راه‌حل مهندسی متداول به شرح زیر ارائه می‌شود:

### راه حل اول: ارسال یک تن پایلوت (Pilot Tone)

فرستنده، علاوه بر سیگنال DSB-SC، یک تن سینوسی کم‌توان دقیقاً در فرکانس حامل  $f_c$  (یا یک هارمونیک مشخص از آن) را نیز ارسال می‌کند. در گیرنده، یک فیلتر باندگذر باریک این تن پایلوت را استخراج کرده و به‌عنوان مرجع فاز و فرکانس برای یک حلقه قفل فاز (Phase-Locked Loop, PLL) استاندارد به کار می‌رود. VCO داخل PLL به‌طور پیوسته فاز و فرکانس خود را با تن پایلوت منطبق می‌کند و خروجی آن به‌عنوان حامل محلی همدوس برای دمودولاسیون استفاده می‌شود. مزیت این روش سادگی و پایداری PLL معمولی است؛ هزینه آن، تخصیص بخشی از توان و پهنای باند کانال به تن پایلوت (کاهش جزئی بازده توان که یکی از مزایای اصلی DSB-SC است) می‌باشد.

### راه حل دوم: حلقه Costas (Costas Loop)

در صورتی که تخصیص توان به تن پایلوت مطلوب نباشد، می‌توان از حلقه Costas استفاده کرد که بدون نیاز به حامل یا تن مرجع مجزا، فاز حامل را از خود سیگنال DSB-SC استخراج می‌کند. این حلقه شامل دو شاخه ضرب‌کننده هم‌فاز ( $I$ ) و متعامد ( $Q$ )، با اختلاف فاز  $90^\circ$  نسبت به شاخه ( $I$ ) است؛ حاصل ضرب خروجی این دو شاخه (پس از فیلترینگ پایین‌گذر) یک سیگنال خطای فاز متناسب با  $\sin(2\Delta\phi)$  تولید می‌کند که برای کنترل VCO محلی در مسیر بازخورد استفاده می‌شود. با همگرایی حلقه، هم  $\Delta\phi \rightarrow 0$  و هم  $\Delta f \rightarrow 0$  (چرا که اختلاف فرکانس معادل یک  $\Delta\phi(t)$  خطی با زمان است که حلقه آن را نیز ردیابی و صفر می‌کند)، بدون هیچ‌گونه تخصیص توان اضافی به تن پایلوت. هزینه این روش، پیچیدگی مداری بیشتر و حساسیت بیشتر به شرایط اولیه و نویز نسبت به PLL معمولی مبتنی بر پایلوت است.

به‌طور خلاصه، هر دو راه‌حل با بازسازی دقیق حامل محلی در گیرنده، اثرات کاهش دامنه (ناشی از  $\Delta\phi$ ) و پدیده تپش (ناشی از  $\Delta f$ ) توضیح داده‌شده در بخش‌های (الف) و (ب) را از بین می‌برند؛ انتخاب بین آن‌ها یک مصالحه مهندسی (trade-off) میان سادگی پیاده‌سازی (تن پایلوت) و بازده توان کامل (حلقه Costas) است.

```

1 clear; clc; close all;
2 Fs = 48000;
3 fc = 15000;
4 sigma2 = 0.01;
5 W = 4000;
6 [m_raw, Fs_read] = audioread('m_t1.mp3');
7 if size(m_raw,2) > 1, m_raw = mean(m_raw,2); end
8 if Fs_read ~= Fs, Fs = Fs_read; end
9 m = m_raw / max(abs(m_raw));
10
11 N = length(m);
12 Ts = 1/Fs;
13 t = (0:N-1)' * Ts;
14 xc = m .* cos(2*pi*fc*t);
15 n = sqrt(sigma2) * randn(N,1);
16 r = xc + n;
17 z = r .* cos(2*pi*fc*t);
18 m_hat = idealLPF(z, Fs, W);
19
20 plotTimeFreq(t, xc, Fs, 'x_c(t) - DSB modulated signal');
21 saveas(gcf, 'fig_dsb_xc.png');
22 plotTimeFreq(t, z, Fs, 'z(t) - receiver mixer output');
23 saveas(gcf, 'fig_dsb_z.png');
24 plotTimeFreq(t, m_hat, Fs, 'm hat(t) - recovered signal');
25 saveas(gcf, 'fig_dsb_mhat.png');
26
27 MSE_ideal = mean((m - m_hat).^2);
28 fprintf('[DSB - ideal case] MSE = %.6g\n', MSE_ideal);
29
30 signal_power = mean(m.^2);
31 error_power = mean((m - m_hat).^2);
32 SNR_dB = 10*log10(signal_power / error_power);
33 fprintf('[DSB - ideal case] Output SNR = %.3f dB\n', SNR_dB);
34
35 phase_errors = [pi/2, pi/4, pi/3];
36 phase_labels = {'pi_2', 'pi_4', 'pi_3'};

```

```

37 MSE_phase = zeros(size(phase_errors));
38 for i = 1:numel(phase_errors)
39     dphi = phase_errors(i);
40     z_p = r .* cos(2*pi*fc*t + dphi);
41     m_hat_p = idealLPF(z_p, Fs, W);
42     MSE_phase(i) = mean((m - m_hat_p).^2);
43     fprintf('DSB - phase error] dphi = %.4f rad -> MSE = %.6g\n', dphi, MSE_phase(i));
44
45     plotTimeFreq(t, m_hat_p, Fs, sprintf('DSB: m hat(t), phase error = %.4f rad', dphi)
46         );
47     saveas(gcf, sprintf('fig_dsb_phase_%s.png', phase_labels{i}));
48 end
49
49 freq_errors = [0.1, 0.01, 0.001] * fc;
50 freq_labels = {'10pct', '1pct', '0p1pct'};
51 MSE_freq = zeros(size(freq_errors));
52 for i = 1:numel(freq_errors)
53     df = freq_errors(i);
54     z_f = r .* cos(2*pi*(fc+df)*t);
55     m_hat_f = idealLPF(z_f, Fs, W);
56     MSE_freq(i) = mean((m - m_hat_f).^2);
57     fprintf('DSB - frequency error] df = %.2f Hz (%.3f%% of fc) -> MSE = %.6g\n', ...
58         df, 100*freq_errors(i)/fc, MSE_freq(i));
59
60     plotTimeFreq(t, m_hat_f, Fs, sprintf('DSB: m hat(t), freq error = %.3f%% of fc',
61         100*freq_errors(i)/fc));
62     saveas(gcf, sprintf('fig_dsb_freq_%s.png', freq_labels{i}));
63 end
64
64 order = 6;
65 Wn = W / (Fs/2);
66 [b, a] = butter(order, Wn, 'low');
67 z0 = r .* cos(2*pi*fc*t);
68 m_hat_butter = filtfilt(b, a, z0);
69 MSE_butter = mean((m - m_hat_butter).^2);
70 fprintf('DSB - Butterworth order %d MSE = %.6g\n', order, MSE_butter);

```

```

71 plotTimeFreq(t, m_hat_butter, Fs, 'DSB: m hat(t), Butterworth LPF');
72 saveas(gcf, 'fig_dsb_butter.png');
73 fprintf('\n=== DSB summary for comparison table ===\n');
74 fprintf('Ideal (no impairment): MSE = %.6g, SNR = %.3f dB\n', MSE_ideal, SNR_dB);
75 fprintf('Phase error (pi/2,pi/4,pi/3): MSE = [%.6g, %.6g, %.6g]\n', MSE_phase);
76 fprintf('Frequency error (10%,1%,0.1%): MSE = [%.6g, %.6g, %.6g]\n', MSE_freq);
77 fprintf('Butterworth filter: MSE = %.6g\n', MSE_butter);

```

Listing 4: DSB-SC part full code (part1-dsb.m)

```

1 function y = idealLPF(x, Fs, W)
2     x = x(:);
3     N = length(x);
4     X = fft(x);
5
6     f = (0:N-1) * (Fs/N);
7     f(f > Fs/2) = f(f > Fs/2) - Fs;
8
9     mask = (abs(f) <= W);
10    X(~mask) = 0;
11
12    y = real(ifft(X));
13 end

```

Listing 5: idealLPF code

```

1 function plotTimeFreq(t, x, Fs, sigName)
2     x = x(:);
3     N = length(x);
4     X = fftshift(fft(x));
5     f = (-N/2 : N/2-1) * (Fs/N);
6
7     figure('Name', sigName, 'NumberTitle', 'off');
8
9     subplot(2,1,1);
10    plot(t, x);
11    xlabel('Time (s)'); ylabel('Amplitude');
12    title(['Time Domain: ' sigName]);
13    grid on;
14
15    subplot(2,1,2);
16    plot(f, abs(X));
17    xlabel('Frequency (Hz)'); ylabel('|X(f)|');
18    title(['Frequency Domain: ' sigName]);
19    grid on;
20 end

```

Listing 6: plotting code

خروجی های کد متلب برای مقادیر SNR و MSE:

[DSB - ideal case] MSE = 0.0226031

[DSB - ideal case] Output SNR = 5.776 dB

[DSB - phase error] dphi = 1.5708 rad -> MSE = 0.0863551

[DSB - phase error] dphi = 0.7854 rad -> MSE = 0.0368902

[DSB - phase error] dphi = 1.0472 rad -> MSE = 0.0491818

[DSB - frequency error] df = 1500.00 Hz (10.000

[DSB - frequency error] df = 150.00 Hz (1.000

[DSB - frequency error] df = 15.00 Hz (0.100

[DSB - Butterworth order 6] MSE = 0.0225528

### ۳- مدولاسیون SSB

تمام خواسته این بخش برای سیگنال پیام m\_al.mp3 انجام شده است. مدولاسیون SSB یکی از روش‌های پیشرفته در مخابرات آنالوگ است که در آن تنها یکی از سایدبندهای سیگنال DSB (بالایی یا پایینی) ارسال شده و سایدبند دیگر به همراه مؤلفه حامل حذف می‌شود. این کار باعث کاهش قابل توجه پهنای باند و افزایش بهره‌وری طیفی می‌گردد. مدل ریاضی سیگنال SSB با استفاده از تبدیل هیلبرت به صورت زیر بیان می‌شود:

$$s(t) = m(t) \cos(2\pi f_c t) \mp \hat{m}(t) \sin(2\pi f_c t) \quad (12)$$

که علامت منفی متناظر با سایدبند بالایی (USB) و علامت مثبت متناظر با سایدبند پایینی (LSB) است. فرکانس حامل  $f_c = 15 \text{ kHz}$  و نویز کانال  $n(t)$  سفید گاوسی با واریانس  $\sigma^2 = 0.01$  در نظر گرفته شده است.

#### ۱.۳ پیاده‌سازی دستی تبدیل هیلبرت

طبق الزام صورت پروژه، تبدیل هیلبرت با تابع آماده hilbert محاسبه نشده است. به جای آن، از تعریف تبدیل هیلبرت در حوزه فرکانس استفاده شده است:

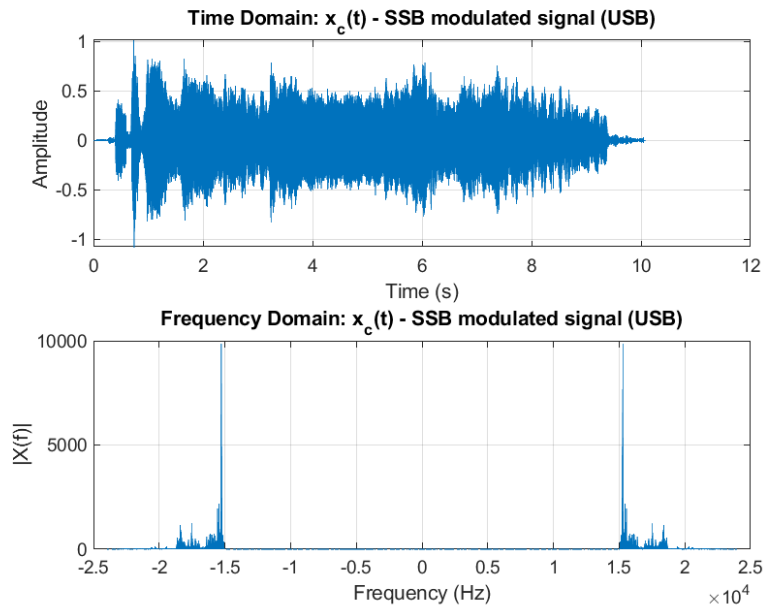
$$H(f) = -j \operatorname{sgn}(f) \quad (13)$$

در حوزه فرکانس، طیف سیگنال تبدیل هیلبرت یافته  $(\hat{M}(f))$  از ضرب مستقیم طیف پیام در پاسخ فرکانسی فیلتر هیلبرت به دست می‌آید:

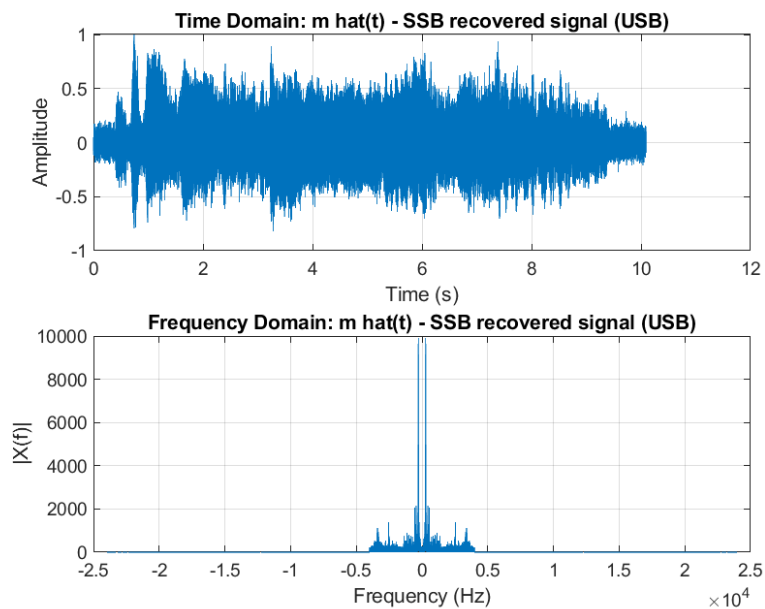
$$\hat{M}(f) = H(f)M(f) = -j \operatorname{sgn}(f)M(f) = \begin{cases} -jM(f), & f > 0 \\ 0, & f = 0 \\ +jM(f), & f < 0 \end{cases} \quad (14)$$

این رابطه نشان می‌دهد فاز مؤلفه‌های فرکانسی مثبت به اندازه  $90^\circ$  عقب و فاز فرکانس‌های منفی به اندازه  $90^\circ$  جلو رانده می‌شود که در تابع manualHilbert.m دقیقاً به همین صورت پیاده‌سازی شده است. یعنی طیف سیگنال با fft محاسبه، در تابع انتقال  $H(f)$  ضرب و سپس با ifft به حوزه زمان بازگردانده می‌شود (تابع manualHilbert.m). دمودولاتور دقیقاً مشابه بخش DSB است.

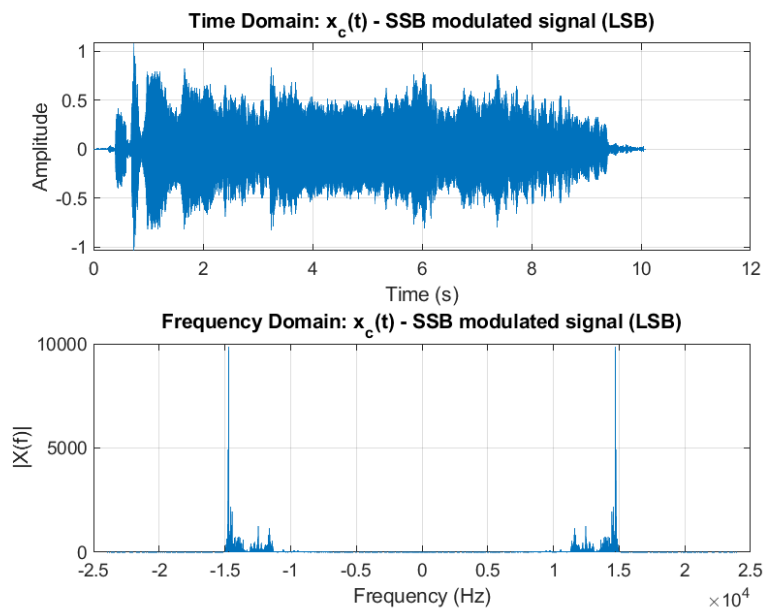
### ۲.۳ سوال ۱: نمایش سیگنال‌های مدوله و بازیابی‌شده



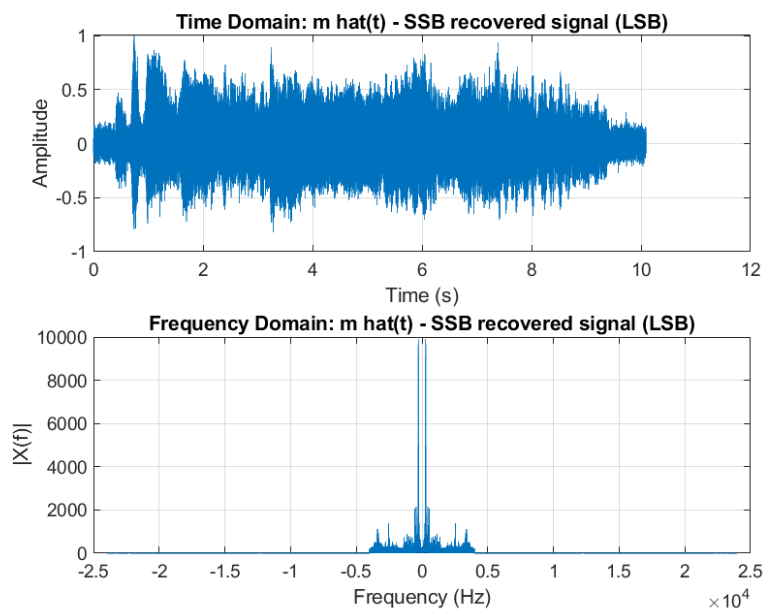
شکل ۲۹: سیگنال مدوله‌شده SSB-USB در حوزه زمان و فرکانس



شکل ۳۰: سیگنال بازیابی‌شده SSB-USB



شکل ۳۱: سیگنال مدوله شده SSB-LSB در حوزه زمان و فرکانس



شکل ۳۲: سیگنال بازیابی شده SSB-LSB



با مقایسه شکل‌های ۲۹ و ۳۱ در حوزه فرکانس می‌توان مشاهده کرد که طیف USB تنها در باند  $f_c$  تا  $f_c + W$  و طیف LSB تنها در باند  $f_c - W$  تا  $f_c$  انرژی دارد؛ برخلاف DSB که طیف پیام به‌طور کامل و متقارن حول  $f_c$  تکرار می‌شد، در نتیجه پهنای باند مصرفی SSB دقیقاً نصف DSB است.

### ۳.۳ سوال ۲ و ۳: محاسبه MSE و SNR

معیار MSE طبق رابطه (۸) و SNR طبق  $\text{SNR}_{\text{dB}} = 10 \log_{10}(P_m/P_{\text{err}})$  برای هر دو حالت محاسبه شده‌اند. مقادیر عددی در جدول ۲ گزارش شده است.

جدول ۲: مقایسه MSE و SNR برای SSB-USB و SSB-LSB

| حالت          | MSE        | SNR (dB) |
|---------------|------------|----------|
| USB (ایده‌آل) | 0.00339523 | 10.881   |
| LSB (ایده‌آل) | 0.00339524 | 10.881   |

### ۴.۳ تکرار سوال ۳: تحلیل اختلالات برای SSB

سه اختلال بخش قبل (اختلاف فاز، اختلاف فرکانس، فیلتر باترورث) برای سیگنال SSB نیز – با تبدیل هیلبرت دستی – بررسی شده‌اند. نماینده انتخاب‌شده برای این تحلیل، حالت USB است؛ به دلیل تقارن ریاضی مسئله، استدلال برای LSB نیز عیناً برقرار است.

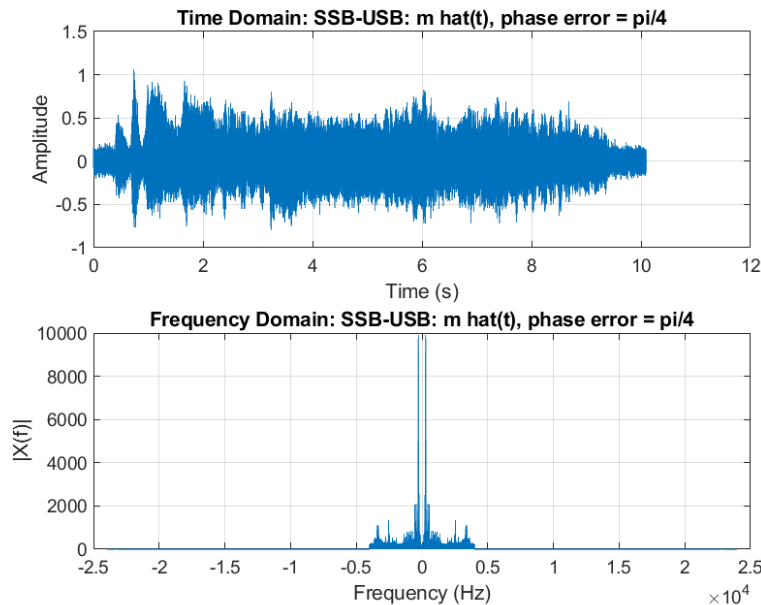
#### ۱.۴.۳ (الف) اختلاف فاز اسیلاتور

با ضرب رابطه (۱۲) (حالت USB) در  $\cos(2\pi f_c t + \Delta\phi)$  و استفاده از اتحاد حاصل ضرب کسینوس‌ها/سینوس‌ها، پس از LPF ایده‌آل داریم:

$$z_{\text{LPF}}(t) = \frac{1}{2} \left[ m(t) \cos(\Delta\phi) + \hat{m}(t) \sin(\Delta\phi) \right] \quad (15)$$

این نتیجه با حالت DSB اساساً متفاوت است: در DSB، اختلاف فاز صرفاً دامنه سیگنال بازیابی‌شده را با ضریب  $\cos(\Delta\phi)$  کاهش می‌داد (رابطه (۴) در بخش ۲)، بدون هیچ اعوجاج شکل موجی. اما در SSB، طبق رابطه (۱۵)، بخشی از تبدیل هیلبرت پیام  $\hat{m}(t)$  نیز با وزن  $\sin(\Delta\phi)$  وارد سیگنال بازیابی‌شده می‌شود؛

این پدیده معادل یک اعوجاج فاز همه‌گذر (all-pass phase distortion) فرکانسی است که کیفیت طنین صدا را تغییر می‌دهد، نه صرفاً دامنه آن را. به همین دلیل، مدولاسیون SSB نسبت به خطای فاز اسیلاتور، حساسیت بیشتری از DSB دارد.



شکل ۳۳: سیگنال بازیابی‌شده SSB-USB با اختلاف فاز  $\Delta\phi = \pi/4$  (نماینده)

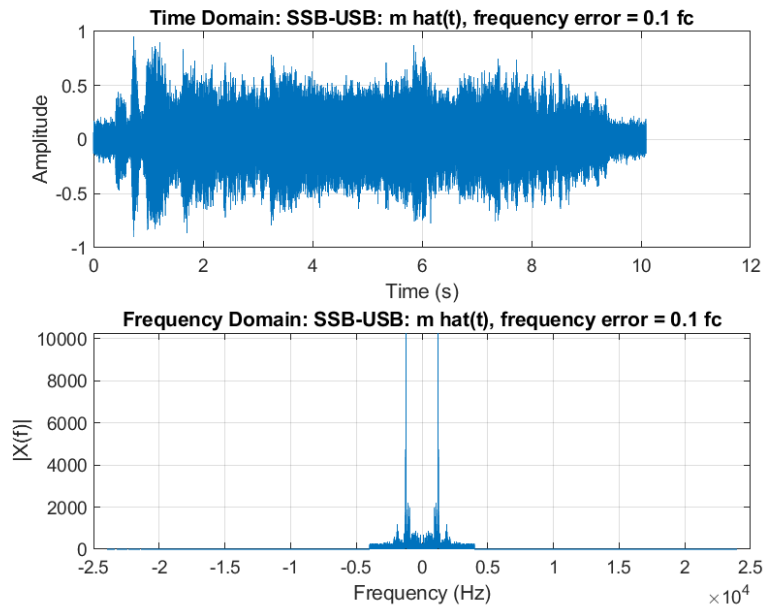
### ۲.۴.۳ (ب) اختلاف فرکانس اسیلاتور

با استدلال مشابه برای اختلاف فرکانس  $\Delta f$ ، پس از LPF داریم:

$$z_{\text{LPF}}(t) = \frac{1}{\sqrt{2}} \left[ m(t) \cos(2\pi \Delta f t) + \hat{m}(t) \sin(2\pi \Delta f t) \right] = \frac{1}{\sqrt{2}} \operatorname{Re} \left\{ [m(t) + j\hat{m}(t)] e^{-j2\pi \Delta f t} \right\} \quad (16)$$

یعنی سیگنال بازیابی‌شده دقیقاً معادل سیگنال تحلیلی (analytic signal) پیام است که با فرکانس  $\Delta f$  جابه‌جا شده است. بنابراین، برخلاف DSB که اختلاف فرکانس باعث پدیده ضرب‌شدگی (beat) دوره‌ای دامنه می‌شد، در SSB کل طیف پیام بازیابی‌شده به اندازه  $\Delta f$  جابه‌جا می‌شود.

این پدیده در رادیوهای SSB واقعی به صورت تغییر طنین صدا (ثُن فلزی/نامفهوم) هنگام عدم تطابق دقیق اسیلاتورهای فرستنده و گیرنده شناخته شده است – برخلاف افت تپشی (tremolo) قابل شنیدن در DSB.



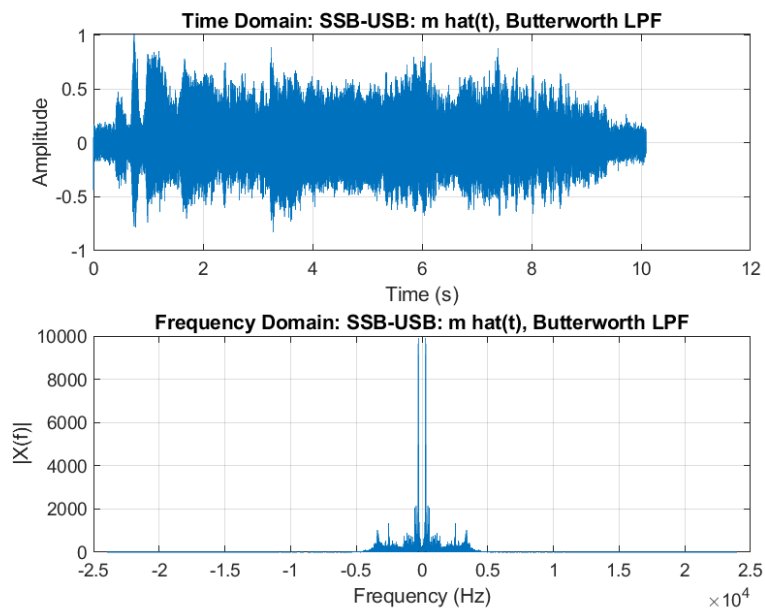
شکل ۳۴: سیگنال بازیابی‌شده SSB-USB با اختلاف فرکانس  $\Delta f = 0.1 f_c$  (نماینده)

### ۳.۴.۳ (ج) فیلتر باترورث به جای LPF ایده‌آل

مشابه بخش DSB، فیلتر ایده‌آل با فیلتر باترورث مرتبه ۶ جایگزین شد (butter به همراه filtfilt). از آنجا که این جایگزینی صرفاً روی مرحله فیلترسازی باند پایه اثر می‌گذارد و مستقل از ساختار مدولاسیون (SSB یا DSB) است، دلیل افزایش جزئی MSE برای SSB دقیقاً مشابه بخش ۲ است: ناحیه گذار غیرایده‌آل فیلتر باعث تضعیف جزئی لبه باند پیام و نشت محدود انرژی باند حذفی می‌شود.

جدول ۳: خلاصه مقایسه MSE برای SSB-USB در حالات مختلف اختلال

| MSE        | حالت                   |
|------------|------------------------|
| 0.0864465  | $\Delta\phi = \pi/2$   |
| 0.0278025  | $\Delta\phi = \pi/4$   |
| 0.0449943  | $\Delta\phi = \pi/3$   |
| 0.086648   | $\Delta f = 0.1 f_c$   |
| 0.0867438  | $\Delta f = 0.01 f_c$  |
| 0.0852808  | $\Delta f = 0.001 f_c$ |
| 0.00319119 | فیلتر باترورث مرتبه ۶  |



شکل ۳۵: سیگنال بازایی‌شده SSB-USB با فیلتر باترورث مرتبه ۶

```

1 function mh = manualHilbert(m, Fs)
2     m = m(:);
3     N = length(m);
4     M = fft(m);
5
6     f = (0:N-1) * (Fs/N);
7     f(f > Fs/2) = f(f > Fs/2) - Fs;
8
9     H = -1i * sign(f(:));
10    H(f == 0) = 0;
11    Mh = M .* H;
12    mh = real(ifft(Mh));
13 end

```

Listing 7: manual hilbert function code

```

1 clear; clc; close all;
2 Fs = 48000;
3 fc = 15000;
4 sigma2 = 0.01;
5 W = 4000;
6 [m_raw, Fs_read] = audioread('m_a1.mp3');
7 if size(m_raw,2) > 1, m_raw = mean(m_raw,2); end
8 if Fs_read ~= Fs, Fs = Fs_read; end
9 m = m_raw / max(abs(m_raw));
10
11 N = length(m);
12 Ts = 1/Fs;
13 t = (0:N-1)' * Ts;
14 m_hilb = manualHilbert(m, Fs);
15
16 xc_usb = m .* cos(2*pi*fc*t) - m_hilb .* sin(2*pi*fc*t); % Upper Sideband
17 xc_lsb = m .* cos(2*pi*fc*t) + m_hilb .* sin(2*pi*fc*t); % Lower Sideband
18
19 n = sqrt(sigma2) * randn(N,1);
20
21 sidebands = struct('name', {'USB', 'LSB'}, 'sig', {xc_usb, xc_lsb});
22
23 MSE_all = zeros(1,2);
24 SNR_all = zeros(1,2);
25
26 for k = 1:2
27     r = sidebands(k).sig + n;
28     m_hat = 2 * idealLPF(z, Fs, W);
29
30     plotTimeFreq(t, sidebands(k).sig, Fs, ['x_c(t) - SSB modulated signal (' sidebands(
        k).name ')']);
31     saveas(gcf, ['fig_ssb_' lower(sidebands(k).name) '_xc.png']);
32
33     plotTimeFreq(t, m_hat, Fs, ['m hat(t) - SSB recovered signal (' sidebands(k).name '
        ')']);
34     saveas(gcf, ['fig_ssb_' lower(sidebands(k).name) '_mhat.png']);

```

```

35
36 MSE_all(k) = mean((m - m_hat).^2);
37 signal_power = mean(m.^2);
38 error_power = mean((m - m_hat).^2);
39 SNR_all(k) = 10*log10(signal_power / error_power);
40
41 fprintf('SSB-%s MSE = %.6g , SNR = %.3f dB\n', sidebands(k).name, MSE_all(k),
    SNR_all(k));
42 end
43
44 r_usb = xc_usb + n;
45
46 phase_errors = [pi/2, pi/4, pi/3];
47 MSE_phase_ssb = zeros(size(phase_errors));
48 for i = 1:numel(phase_errors)
49     dphi = phase_errors(i);
50     z_p = r_usb .* cos(2*pi*fc*t + dphi);
51     m_hat_p = 2 * idealLPF(z_p, Fs, W);
52     MSE_phase_ssb(i) = mean((m - m_hat_p).^2);
53     fprintf('SSB-USB - phase error dphi = %.4f rad -> MSE = %.6g\n', dphi,
        MSE_phase_ssb(i));
54 end
55 z_rep_phase = r_usb .* cos(2*pi*fc*t + pi/4);
56 plotTimeFreq(t, 2*idealLPF(z_rep_phase, Fs, W), Fs, 'SSB-USB: m hat(t), phase error =
    pi/4');
57 saveas(gcf, 'fig_ssb_phase_rep.png');
58
59 freq_errors = [0.1, 0.01, 0.001] * fc;
60 MSE_freq_ssb = zeros(size(freq_errors));
61 for i = 1:numel(freq_errors)
62     df = freq_errors(i);
63     z_f = r_usb .* cos(2*pi*(fc+df)*t);
64     m_hat_f = 2 * idealLPF(z_f, Fs, W);
65     MSE_freq_ssb(i) = mean((m - m_hat_f).^2);
66     fprintf('SSB-USB - frequency error df = %.2f Hz -> MSE = %.6g\n', df,
        MSE_freq_ssb(i));

```

```

67 end
68 z_rep_freq = r_usb .* cos(2*pi*(fc+freq_errors(1))*t);
69 plotTimeFreq(t, 2*idealLPF(z_rep_freq, Fs, W), Fs, 'SSB-USB: m hat(t), frequency error
    = 0.1 fc');
70 saveas(gcf, 'fig_ssb_freq_rep.png');
71
72 order = 6;
73 Wn = W / (Fs/2);
74 [b, a] = butter(order, Wn, 'low');
75
76 z0 = r_usb .* cos(2*pi*fc*t);
77 m_hat_butter_ssb = 2 * filtfilt(b, a, z0);
78 MSE_butter_ssb = mean((m - m_hat_butter_ssb).^2);
79 fprintf('SSB-USB - Butterworth order %d MSE = %.6g\n', order, MSE_butter_ssb);
80 plotTimeFreq(t, m_hat_butter_ssb, Fs, 'SSB-USB: m hat(t), Butterworth LPF');
81 saveas(gcf, 'fig_ssb_butter.png');
82
83 fprintf('\n=== SSB summary for comparison table ===\n');
84 fprintf('USB ideal: MSE = %.6g, SNR = %.3f dB\n', MSE_all(1), SNR_all(1));
85 fprintf('LSB ideal: MSE = %.6g, SNR = %.3f dB\n', MSE_all(2), SNR_all(2));
86 fprintf('USB phase error (pi/2,pi/4,pi/3): MSE = [%.6g, %.6g, %.6g]\n', MSE_phase_ssb)
    ;
87 fprintf('USB frequency error (10%,1%,0.1%): MSE = [%.6g, %.6g, %.6g]\n',
    MSE_freq_ssb);
88 fprintf('USB Butterworth filter: MSE = %.6g\n', MSE_butter_ssb);

```

Listing 8: full code of SSB part

خروجی کد متلب برای مقادیر SNR و MSE:

[SSB-USB] MSE = 0.00339523 , SNR = 10.881 dB

[SSB-LSB] MSE = 0.00339524 , SNR = 10.881 dB

[SSB-USB - phase error] dphi = 1.5708 rad -> MSE = 0.0864465

[SSB-USB - phase error] dphi = 0.7854 rad -> MSE = 0.0278025

[SSB-USB - phase error] dphi = 1.0472 rad -> MSE = 0.0449943

[SSB-USB - frequency error] df = 1500.00 Hz -> MSE = 0.086648

[SSB-USB - frequency error] df = 150.00 Hz -> MSE = 0.0867438

[SSB-USB - frequency error] df = 15.00 Hz -> MSE = 0.0852808

[SSB-USB - Butterworth order 6] MSE = 0.00319119



## ۴- مدولاسیون FM (Simulink)

در این بخش، برخلاف بخش‌های قبلی، مدولاسیون FM با استفاده از Simulink در MATLAB پیاده‌سازی و نمایش داده می‌شود، زیرا ماهیت غیرخطی و انتگرال‌گیری پیوسته این مدولاسیون، پیاده‌سازی بلوکی و شبیه‌سازی زمان‌پیوسته را نسبت به کدنویسی مستقیم مناسب‌تر می‌سازد.

مدولاسیون فرکانسی (FM) یکی از روش‌های مدولاسیون آنالوگ غیرخطی است که در آن اطلاعات سیگنال پیام در تغییرات فرکانس سیگنال حامل منتقل می‌شود، در حالی که دامنه حامل ثابت باقی می‌ماند. این ویژگی باعث می‌شود FM نسبت به نویزهای دامنه‌ای مقاومت بیشتری داشته باشد. مدل ریاضی سیگنال FM به صورت زیر بیان می‌شود:

$$s(t) = A_c \cos\left(2\pi f_c t + 2\pi k_f \int_0^t m(\lambda) d\lambda\right) \quad (17)$$

که فرکانس لحظه‌ای آن برابر است با:

$$f(t) = f_c + k_f m(t) \quad (18)$$

که نشان می‌دهد فرکانس سیگنال خروجی متناسب با مقدار لحظه‌ای سیگنال پیام تغییر می‌کند. پارامترهای مدل: سیگنال پیام سینوسی با فرکانس  $f_m = 2 \text{ kHz}$  (بلوک Sine Wave)، فرکانس حامل  $f_c = 10 \text{ kHz}$ ، ضریب حساسیت فرکانسی  $k_f = 2500$ ، و نویز کانال AWGN با واریانس  $\sigma^2 = 0.01$  که بلافاصله پس از مدولاتور از بلوک AWGN Channel اعمال می‌شود.

### ۱.۴ سوال ۱: مدولاتور مبتنی بر بلوک VCO

بلوک Continuous-Time VCO در Simulink دقیقاً رفتار رابطه (۱۷) را پیاده‌سازی می‌کند: ورودی این بلوک یک سیگنال کنترل ولتاژ پیوسته (در اینجا  $m(t)$ ، خروجی بلوک Sine Wave) است، و خروجی آن یک موج سینوسی با فرکانس لحظه‌ای متناسب با ورودی – طبق رابطه (۱۸) – تولید می‌کند. پارامترهای اصلی بلوک عبارت‌اند از:

• **Quiescent Frequency:** فرکانس مرکزی نوسانگر در غیاب ورودی، که برابر  $f_c = 10 \text{ kHz}$  تنظیم می‌شود.

• **Input Sensitivity (بهره VCO):** ضریب تبدیل ولتاژ ورودی به انحراف فرکانس، که معادل  $k_f = 2500 \text{ Hz/V}$  است.

داخل بلوک، سیگنال ورودی به‌طور پیوسته انتگرال‌گیری شده و به‌عنوان فاز لحظه‌ای یک نوسانگر کسینوسی به‌کار می‌رود؛ این عملیات دقیقاً معادل انتگرال‌گیری صریح  $\int_0^t m(\lambda) d\lambda$  در رابطه (۱۷) است، بدون نیاز به پیاده‌سازی دستی انتگرال‌گیر. از آن‌جا که بلوک از نوع Continuous-Time است (نه Discrete-Time)، نیازی به انتخاب دستی گام زمانی نمونه‌برداری برای مدولاتور نیست؛ حل‌گر (solver) پیوسته سیمولینک به‌طور خودکار گام‌های زمانی کافی برای نمایش دقیق نوسان حامل با فرکانس بالا انتخاب می‌کند.

## ۲.۴ سوال ۲: جایگزینی آشکارساز فاز با بلوک ضرب‌کننده (Matrix Multiply)

یک دمودولاتور کلاسیک PLL (Phase-Locked Loop) از سه بخش تشکیل می‌شود: آشکارساز فاز (Phase Detector)، فیلتر پایین‌گذر حلقه (Loop Filter) و یک VCO در مسیر بازخورد که خروجی آن با سیگنال ورودی مقایسه می‌شود. چون بلوکی با نام Phase Detector در کتابخانه Simulink وجود ندارد، از یک بلوک ضرب‌کننده (معادل Matrix Multiply)، در مدل با برچسب  $x$  نمایش داده شده) برای ضرب نمونه‌به‌نمونه سیگنال دریافتی  $r(t)$  در خروجی VCO داخلی حلقه  $y_{VCO}(t)$  استفاده می‌شود. برای اثبات این‌که این ضرب معادل عملکرد یک آشکارساز فاز آنالوگ است، دو سیگنال را با فاز لحظه‌ای  $\theta_r(t)$  و  $\theta_v(t)$  در نظر می‌گیریم:

$$e(t) = r(t) \cdot y_{VCO}(t) = A_c A_v \cos(\theta_r(t)) \cos(\theta_v(t)) \quad (۱۹)$$

با استفاده از اتحاد حاصل‌ضرب کسینوس‌ها:

$$e(t) = \frac{A_c A_v}{2} \left[ \cos(\theta_r(t) - \theta_v(t)) + \cos(\theta_r(t) + \theta_v(t)) \right] \quad (۲۰)$$

پس از عبور از فیلتر پایین‌گذر حلقه، جمله فرکانس بالا (حول  $2f_c$ ) حذف شده و باقی می‌ماند:

$$e_{LPF}(t) = \frac{A_c A_v}{2} \cos(\Delta\theta(t)), \quad \Delta\theta(t) = \theta_r(t) - \theta_v(t) \quad (۲۱)$$

در حالت قفل‌شدگی حلقه (lock)، VCO داخلی با اختلاف فاز درجا  $90^\circ$  نسبت به سیگنال ورودی تنظیم می‌شود (ویژگی استاندارد PLL مبتنی بر ضرب‌کننده آنالوگ، لازم برای آنکه منحنی مشخصه آشکارساز فاز نزدیک نقطه تعادل تقریباً خطی و فرد باشد)، بنابراین  $\Delta\theta(t) = \pi/2 + \phi_e(t)$  که  $\phi_e(t)$  خطای فاز کوچک باقی‌مانده است. با جایگذاری:

$$e_{LPF}(t) = \frac{A_c A_v}{2} \cos\left(\frac{\pi}{2} + \phi_e(t)\right) = -\frac{A_c A_v}{2} \sin(\phi_e(t)) \approx -\frac{A_c A_v}{2} \phi_e(t) \quad (۲۲)$$

که برای خطای فاز کوچک، تقریب خطی  $\sin(\phi_e) \approx \phi_e$  معتبر است. این سیگنال خطا، هم به عنوان ورودی کنترل VCO (برای رانده‌شدن حلقه به سمت قفل‌شدگی) و هم به عنوان خروجی دمدوله‌شده استفاده می‌شود.

رفتار فرکانس لحظه‌ای VCO موجود در مسیر بازخورد حلقه قفل فاز (PLL) به صورت زیر تعریف می‌شود:

$$\frac{d\theta_v(t)}{dt} = 2\pi f_c + 2\pi k_v e_{\text{LPF}}(t) \quad (23)$$

که در آن  $k_v$  بهره فرکانسی VCO داخلی است. در حالت قفل کامل (Lock)، فرکانس لحظه‌ای VCO داخلی به طور پیوسته فرکانس لحظه‌ای سیگنال ورودی یعنی  $f(t) = f_c + k_f m(t)$  را دنبال می‌کند  $(\frac{d\theta_v}{dt} \approx \frac{d\theta_r}{dt})$ . با مساوی قرار دادن این دو رابطه خواهیم داشت:

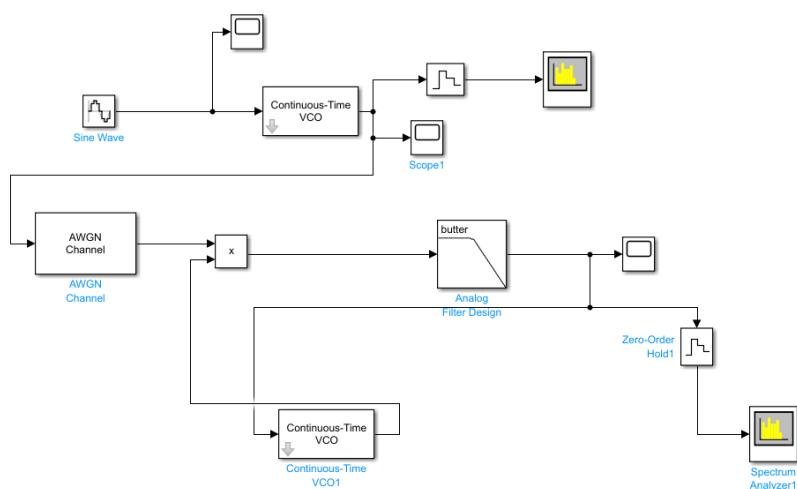
$$2\pi f_c + 2\pi k_v e_{\text{LPF}}(t) = 2\pi f_c + 2\pi k_f m(t) \implies e_{\text{LPF}}(t) = \frac{k_f}{k_v} m(t) \quad (24)$$

این رابطه به وضوح نشان می‌دهد که خروجی فیلتر حلقه  $(e_{\text{LPF}}(t))$  مستقیماً و به صورت خطی با سیگنال پیام اصلی  $m(t)$  متناسب است و نیازی به بلوک مشتق‌گیر مجزا برای دمودولاسیون نیست.

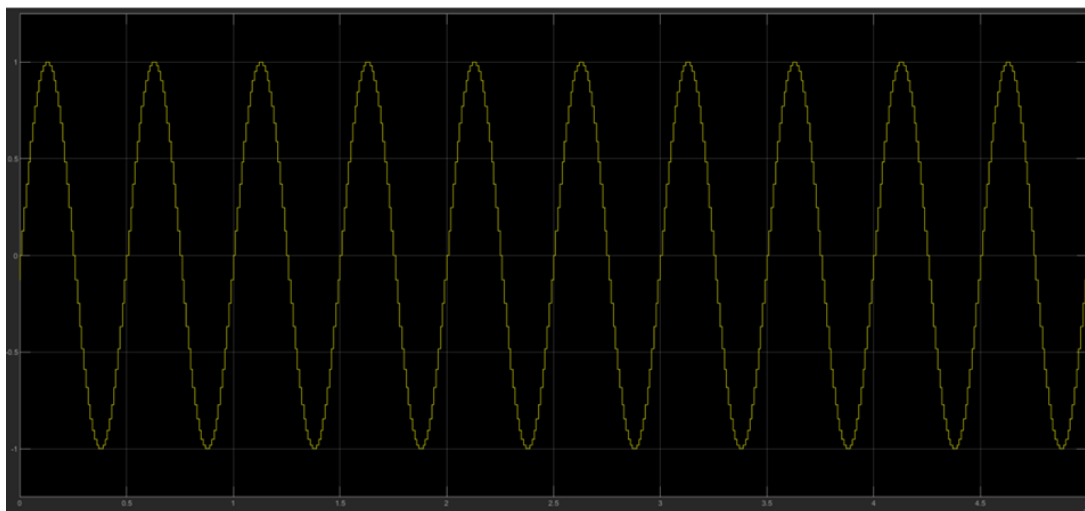
### ۳.۴ سوال ۳: ساختار مدل و نتایج

مدل پیاده‌سازی شده (شکل ۳۶) شامل زنجیره زیر است: بلوک Sine Wave (پیام ۲ kHz)  $\rightarrow$  بلوک Continuous-Time VCO (مدولاتور، طبق پارامترهای بخش ۱). سیگنال پیام خام پیش از VCO با یک Scope جداگانه نمایش داده شده (شکل ۳۷) و خروجی مدولاتور نیز با Scope1 در حوزه زمان (شکل ۳۸) و از مسیر Zero-Order Hold  $\rightarrow$  Spectrum Analyzer در حوزه فرکانس (شکل ۴۰) نمایش داده شده است.

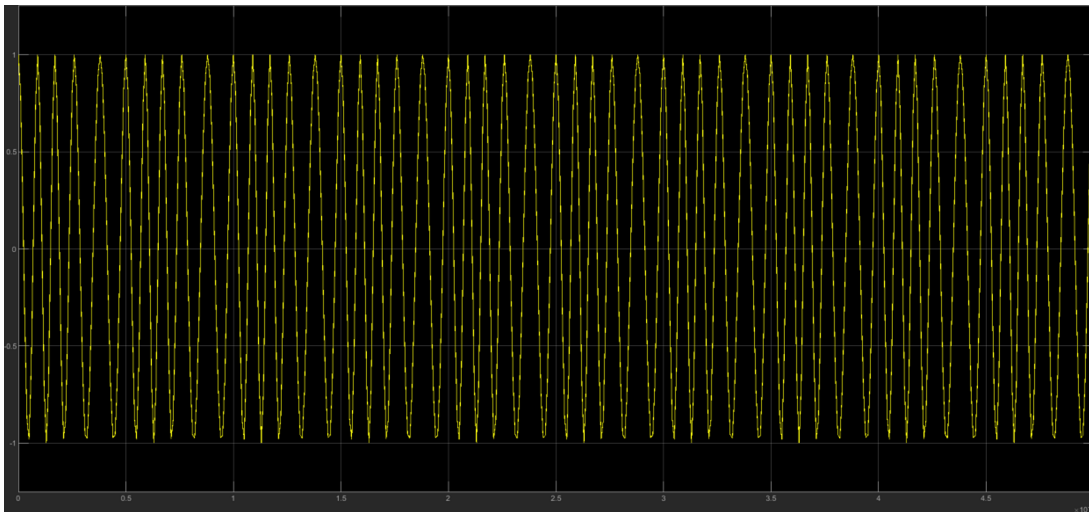
سیگنال مدوله شده سپس وارد بلوک AWGN Channel  $(\sigma^2 = 0.01)$  می‌شود. داخل حلقه PLL: سیگنال دریافتی نویزی و خروجی یک Continuous-Time VCO دوم (VCO1، در مسیر بازخورد) به بلوک ضرب‌کننده  $x$  وارد می‌شوند. خروجی این بلوک از یک فیلتر پایین‌گذر واقعی (بلوک Analog Filter Design با پیکربندی butter، معادل همان فیلتر باترورث استفاده شده در بخش‌های SSB/DSB) عبور می‌کند. خروجی این فیلتر همزمان (۱) با یک Scope در حوزه زمان (شکل ۳۹) و از مسیر Zero-Order Hold1  $\rightarrow$  Spectrum Analyzer1 در حوزه فرکانس (شکل ۴۱) نمایش داده می‌شود، و (۲) به عنوان ورودی کنترل VCO1 باز می‌گردد تا حلقه PLL بسته شود.



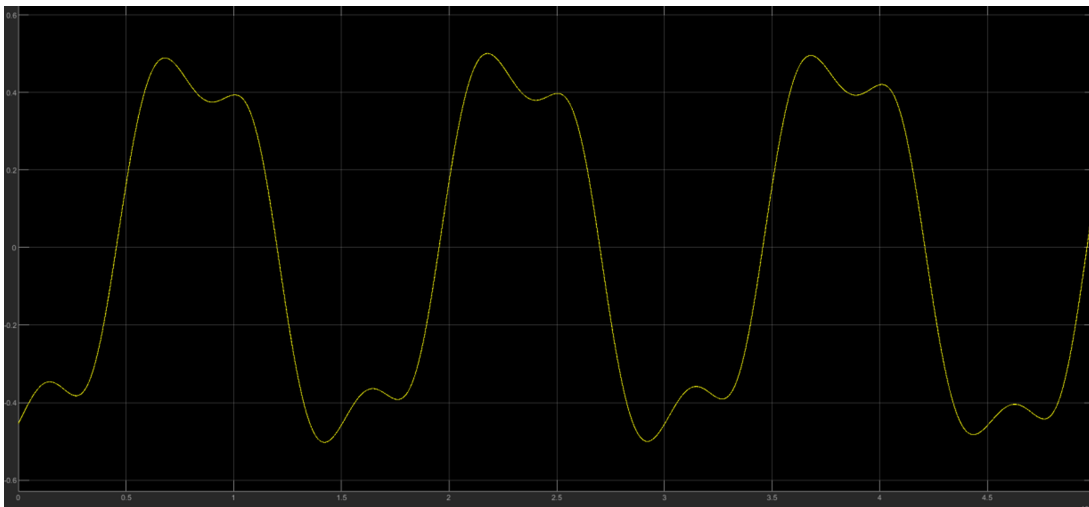
شکل ۳۶: بلوک دیاگرام کامل مدل Simulink



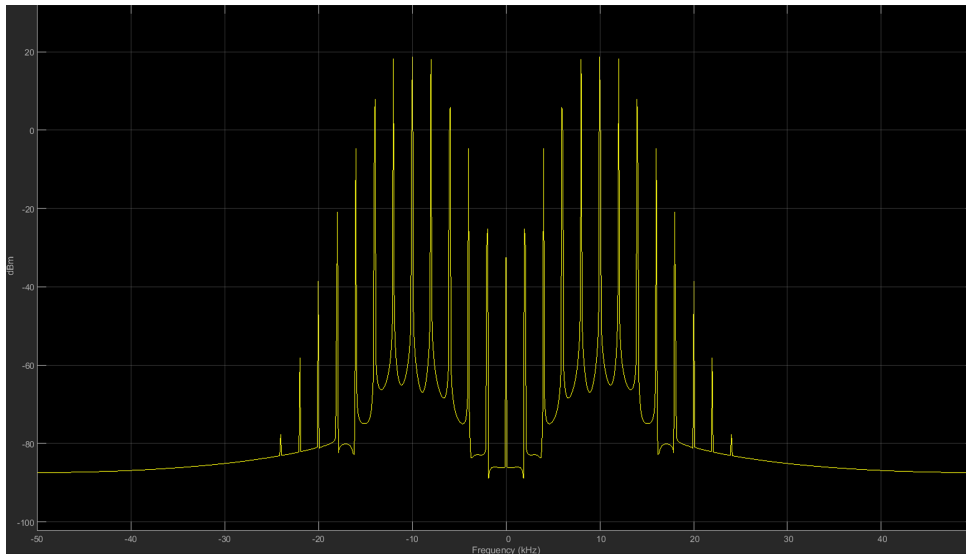
شکل ۳۷: سیگنال پیام  $m(t)$  در حوزه زمان (خروجی بلوک Sine Wave،  $f_m = 2$  کیلوهرتز)، پیش از ورود به VCO.



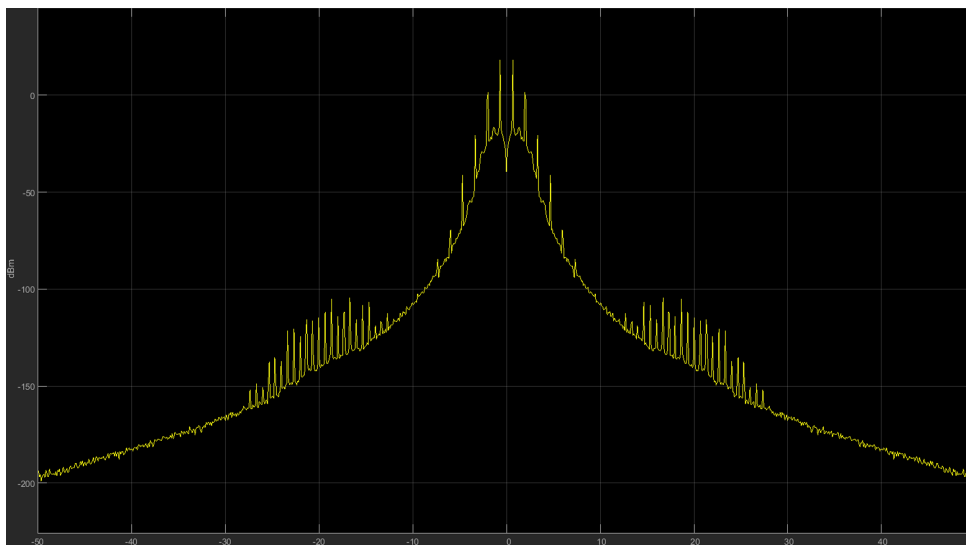
شکل ۳۸: سیگنال مدوله‌شده FM در حوزه زمان (خروجی Scope1، بعد از VCO). نوسان بسیار متراکم‌تر نسبت به شکل ۳۷ ناشی از فرکانس حامل بالا ( $f_c = 10$  کیلوهرتز) و انحراف فرکانس  $\Delta f = 2500$  هرتز است.



شکل ۳۹: سیگنال دمدوله‌شده  $\hat{m}(t)$  در حوزه زمان، پس از عبور از فیلتر باترورث انتهای حلقه PLL. دامنه بازیابی‌شده (حدود ۵/۰) نسبت به دامنه واحد پیام اصلی مقیاس‌نشده است، چون ضریب بهره حلقه (حاصل‌ضرب بهره آشکارساز فاز، فیلتر و VCO) نرمالیزه نشده؛ شکل کلی موج با یک تأخیر/اعوجاج جزئی ناشی از فیلتر و ثابت‌زمانی قفل‌شدگی حلقه، مطابق پیام اصلی است.



شکل ۴۰: خروجی بلوک Spectrum Analyzer در حوزه فرکانس برای سیگنال FM مدوله‌شده، قبل از کانال AWGN. طیف حول  $\pm f_c = \pm 10$  کیلوهرتز متمرکز است که با پیش‌بینی رابطه Carson (۵۵۰۰ تا ۱۴۵۰۰ هرتز، طبق سوال ۴) هم‌خوانی معقولی دارد.



شکل ۴۱: خروجی بلوک Spectrum Analyzer1 در حوزه فرکانس برای سیگنال دمدوله‌شده (خروجی نهایی حلقه PLL). برخلاف شکل ۴۰، طیف حول باند پایه ( $f = 0$ ) متمرکز شده که تأییدکننده صحت عملکرد دمدولاتور است؛ دو برآمدگی جانبی ضعیف نزدیک  $\pm 20$  تا  $\pm 25$  کیلوهرتز نیز قابل مشاهده‌اند که احتمالاً ناشی از نشت جزئی مؤلفه فرکانس دوبرابر ( $2f_c$ ) پیش از فیلترسازی کامل یا آرتیفکت‌های نمونه‌برداری بلوک Zero-Order Hold هستند.

## ۴.۴ سوال ۴: رابطه Carson

قاعده تجربی Carson پهنای باند مؤثر سیگنال FM را تخمین می‌زند:

$$B \approx 2(\Delta f + f_m) = 2(k_f \max |m(t)| + f_m) \quad (25)$$

که  $\Delta f = k_f \max |m(t)|$  انحراف فرکانس بیشینه (frequency deviation) و  $f_m$  فرکانس سیگنال پیام است. با پارامترهای پروژه  $k_f = 2500$ ،  $\max |m(t)| = 1$  برای موج سینوسی با دامنه واحد،  $f_m = 2 \text{ kHz}$ ، انحراف فرکانس بیشینه برابر:

$$\Delta f = k_f \cdot 1 = 2500 \text{ Hz} \quad (26)$$

و ضریب مدولاسیون FM:

$$\beta = \frac{\Delta f}{f_m} = \frac{2500}{2000} = 1.25 \quad (27)$$

از آنجا که  $\beta > 1$ ، سیگنال در رژیم FM پهن‌بند (wideband FM) قرار دارد (برخلاف FM باریک‌بند که در آن  $\beta \ll 1$  و  $B \approx 2f_m$  می‌شود). پهنای باند تخمینی طبق رابطه (25):

$$B \approx 2(2500 + 2000) = 9000 \text{ Hz} \quad (28)$$

یعنی انتظار می‌رود انرژی محسوس طیف در بازه تقریبی  $f_c \pm B/2 = 10000 \pm 4500$ ، یعنی از  $5500 \text{ Hz}$  تا  $14500 \text{ Hz}$ ، متمرکز باشد. با مشاهده شکل ۴۰، طیف واقعی سیگنال مدوله‌شده در بازه‌ای تقریباً مشابه (حدود ۵ تا ۱۹ کیلوهرتز حول  $f_c = 10$  کیلوهرتز) متمرکز است؛ اختلاف جزئی با پیش‌بینی نظری (۹ کیلوهرتز) طبیعی است، چون رابطه Carson تنها پهنای باندی را تخمین می‌زند که حدود ۹۸٪ توان سیگنال را در بر می‌گیرد، نه یک برش دقیق و قطعی، و طیف واقعی همواره دنباله‌های نمایی با توان کم فراتر از این بازه دارد.

## ۵.۴ سوال ۵: برآورد SNR از روی Spectrum Analyzer

برای برآورد تقریبی SNR سیگنال بازیابی‌شده از روی نمودار Spectrum Analyzer، از روش زیر استفاده می‌شود:

۱. توان سیگنال  $P_{\text{signal}}$  به صورت انرژی متمرکز در  $f_m = 2 \text{ kHz}$  (نزدیک پیام اصلی پیام) در خروجی دمدوله‌شده (خوانده می‌شود).

۲. توان نویز  $P_{\text{noise}}$  به صورت سطح میانگین کف طیف (noise floor) در باندهای خارج از  $f_m$  اصلی پیام (اما داخل باند عبور فیلتر حلقه) خوانده می‌شود.

۳. نسبت سیگنال به نویز تقریبی از رابطه زیر به دست می‌آید:

$$\text{SNR}_{\text{dB}} \approx P_{\text{signal, dB}} - P_{\text{floor, dB noise}} \quad (29)$$

لازم به ذکر است که این روش یک برآورد تقریبی است (نه یک اندازه‌گیری دقیق آماری مبتنی بر میانگین‌گیری روی چند اجرا)، اما برای مقایسه کیفی کافی است. طبق ویژگی شناخته‌شده FM (پدیده بهره پردازش یا FM improvement/processing gain)، انتظار می‌رود SNR خروجی به طور قابل توجهی بهتر از SNR کانال ورودی باشد، مشروط بر آن که سیستم بالای آستانه FM (FM threshold) عمل کند؛ در غیر این صورت (در SNR ورودی بسیار پایین)، پدیده click noise در حلقه PLL می‌تواند باعث افت شدید و ناگهانی کیفیت دمدولاسیون شود. با توجه به واریانس نویز کانال نسبتاً کوچک ( $\sigma^2 = 0.01$ ) در این پروژه، سیستم به وضوح بالای آستانه FM عمل می‌کند و بهره پردازش قابل توجهی انتظار می‌رود. با خواندن سطح  $f_m$  اصلی پیام (حدود ۱۵ تا ۲۰ dBm در نزدیکی  $f = 0$ ) و سطح کف نویز پیرامونی (حدود -۸۰ تا -۱۰۰ dBm، با احتساب دو برآمدگی جانبی ضعیف اطراف  $\pm 20$  تا  $\pm 25$  کیلوهرتز) از شکل ۴۱، مقدار تقریبی  $\text{SNR}_{\text{dB}} \approx 100 \text{ dB}$  به دست می‌آید.



```

1 clear; clc;
2
3 fc = 10000;
4 kf = 2500;
5 fm = 2000;
6 Am = 1;
7
8 delta_f = kf * Am;
9 fprintf('Peak frequency deviation: Delta_f = %.1f Hz\n', delta_f);
10
11 %% FM: beta = Delta_f / fm
12 beta = delta_f / fm;
13 fprintf('FM modulation index: beta = %.3f\n', beta);
14
15 %% Carson: B = 2*(Delta_f + fm)
16 B_carson = 2 * (delta_f + fm);
17 fprintf('Carson bandwidth: B = %.1f Hz\n', B_carson);
18
19 %% Spectrum Analyzer
20 f_low = fc - B_carson/2;
21 f_high = fc + B_carson/2;
22 fprintf('Expected significant spectral content: %.1f Hz to %.1f Hz\n', f_low, f_high);
23 fprintf('(Compare this range against the Spectrum Analyzer plot from the Simulink
    model.)\n');

```

Listing 9: سوال Carson باند پهنای عددی محاسبه برای کمک‌ی MATLAB کد

خروجی کد متلب:

Peak frequency deviation: Delta\_f = 2500.0 Hz

FM modulation index: beta = 1.250

Carson bandwidth: B = 9000.0 Hz

Expected significant spectral content: 5500.0 Hz to 14500.0 Hz

(Compare this range against the Spectrum Analyzer plot from the Simulink model.)

## مراجع

- [۱] Sklar, B. Applications, and Fundamentals Communications: Digital ed. nd۲ Prentice Hall, ۲۰۰۱.
- [۲] Moher, M. and Haykin S. Systems, Communication ed. th۵ Wiley, ۲۰۰۹.
- [۳] Salehi, M. and Proakis G. J. Engineering, Systems Communication ed. nd۲ Prentice Hall, ۲۰۰۲.
- [۴] Crilly, B. P. and Carlson B. A. Introduction An Systems: Communication ed. th۵ McGraw-Hill, ۲۰۱۰.
- [۵] MathWorks, "Fast Fourier Transform," MATLAB Documentation. [Online]. Available: <https://www.mathworks.com/help/matlab/ref/fft.html>
- [۶] MathWorks, "fftshift," MATLAB Documentation. [Online]. Available: <https://www.mathworks.com/help/matlab/ref/fftshift.html>
- [۷] MathWorks, "Butterworth Filter Design," MATLAB Documentation. [Online]. Available: <https://www.mathworks.com/help/signal/ref/butter.html>
- [۸] MathWorks, "Continuous-Time Voltage-Controlled Oscillator (VCO) Using Simulink," MATLAB Documentation. [Online]. Available: <https://www.mathworks.com/help/simulink/slref/continuoustimevco.html>